

Deep-Learning vs Classical Machine Learning for Log Anomaly Detection on System Log Dataset: A SIEM-Style Evaluation

by

Munzarin Tabassum

24241210

Nubah Nazifa Hossain

21201717

Md. Zahidul Hasan Zian

21201199

Sumaiya Hossain

24341183

Hasan Mahmud Mazumder

21201480

A thesis submitted to the Department of Computer Science and Engineering
in partial fulfillment of the requirements for the degree of
B.Sc. in Computer Science and Engineering

Department of Computer Science and Engineering
Brac University
June 2026.

© 2026 Authors
All rights reserved.

Declaration

It is hereby declared that

1. The thesis submitted is our own original work while completing degree at Brac University.
2. The thesis does not contain material previously published or written by a third party, except where this is appropriately cited through full and accurate referencing.
3. The thesis does not contain material which has been accepted, or submitted, for any other degree or diploma at a university or other institution.
4. We have acknowledged all main sources of help.

Student's Full Name & Signature:



Munzarin Tabassum
24241210



Nubah Nazifa Hossain
21201717



Md. Zahidul Hasan Zian
21201199



Sumaiya Hossain
24341183



Hasan Mahmud Mazumder
21201480

Approval

The thesis titled “Deep-Learning vs Classical Machine Learning for Log Anomaly Detection on System Log Dataset: A SIEM-Style Evaluation” submitted by

1. Munzarin Tabassum (24241210)
2. Nubah Nazifa Hossain (21201717)
3. Md. Zahidul Hasan Zian (21201199)
4. Sumaiya Hossain (24341183)
5. Hasan Mahmud Mazumder (21201480)

of Spring, 2025 has been accepted as satisfactory in partial fulfillment of the requirement for the degree of B.Sc. in Computer Science and Engineering in June, 2026.

Examining Committee:

Supervisor:
(Member)



Annajiat Alim Rasel

Senior Lecturer

Department of Computer Science and Engineering
Brac University

Co-Supervisor:
(Member)



Muhammad Abdur Rahman

Senior Adjunct Lecturer

Department of Computer Science and Engineering
Brac University

Thesis Coordinator:
(Member)

Md. Golam Rabiul Alam
Professor
Department of Computer Science and engineering
Brac University

Head of Department:
(Chair)

Dr. Sadia Hamid Kazi
Associate Professor & Chairperson
Department of Computer Science and Engineering
Brac University

Ethics Statement

This study uses publicly available log datasets (HDFS via LogHub) that are provided for research and benchmarking purposes. No information concerning human subjects is collected and no personally identifiable information is processed. Results are presented in summary format with standard performance metrics and SIEM style alerting without revealing any sensitive system information. The evaluations are conducted offline in a controlled environment and do not involve deployment on live systems. The study is mindful of responsible security practice: findings are presented to improve detection and reduce false alerts, and the work does not provide instructions for attacking systems or avoiding detection by security systems.

Abstract

Log anomaly detection (LAD) is vital for modern computing systems, where the large volume of generated logs makes manual analysis impractical. Deep learning has demonstrated strong capabilities in modelling complex log patterns; however, most existing studies evaluate these approaches using traditional metrics that do not fully capture the operational effectiveness of a model. This limitation is particularly relevant in Security Information and Event Management (SIEM) environments, where high false positive rates increase alert volume, contribute to analyst fatigue, and reduce the effectiveness of security operations. This thesis compares four log anomaly detection models (LSTM, Autoencoder, LSTM Autoencoder and Random Forest), representing three anomaly detection paradigms (sequence-based, reconstruction-based, and frequency-based), using a SIEM-oriented evaluation framework. By identical pre-processing, blockwise split, and class imbalance handling and thresholding strategies, model performance was assessed using precision, recall, F1-score, false positive rate, specificity, alert volume with training time and inference throughput. The results show that anomaly characteristics have a significant impact on model performance, with the exploratory data analysis confirming that anomalies in HDFS_v1 are mainly frequency-based. Random Forest with random oversampling performed the best, while from deep learning the LSTM-AE achieved best anomaly detection under specific settings. The results support the SIEM-centric evaluation and indicate that evaluations further than benchmark metrics are needed to reveal operational costs such as alert volume and false positive rate.

Keywords: Anomaly, Deep Learning, HDFS, Log, Machine Learning, SIEM

Acknowledgement

Firstly, we would like to express our deepest gratitude to the Almighty Allah, by whose grace and mercy we were able to complete our thesis successfully.

We took inspiration from influential work of Michael R.Lyu (CUHK, AAAS Fellow) and Prof. Feifei Li (ACM Fellow at Alibaba Cloud).

We would also like to extend our heartfelt appreciation to our respected supervisor, Annajiat Alim Rasel Sir, and our co-supervisor, Muhammad Abdur Rahman Sir, for their continuous support, encouragement, and valuable guidance throughout the course of this research. Their constructive feedback and deep insights played a crucial role in shaping the direction and quality of this work.

Table of Contents

Declaration	i
Approval	i
Ethics Statement	iv
Abstract	v
Acknowledgment	vi
Table of Contents	vii
List of Figures	x
List of Tables	xii
List of Equations	xiii
Nomenclature	xv
1 Introduction	1
1.1 Background	1
1.2 Rationale of the Study or Motivation	2
1.3 Problem Statement	2
1.4 Objective	3
1.5 Methodology in Brief	3
1.6 Scopes and Challenges	4
2 Literature Review	5
2.1 Preliminaries	5
2.1.1 System Logs and Log Anomalies	5
2.1.2 SIEM, SOC and Why ‘Operational’ Evaluation Matters	6
2.1.3 Common Datasets and Benchmark Logs	7
2.2 Review of Existing Research	7
2.3 Summary of Key Findings	11

2.3.1	Key Results and Trends	11
2.3.2	Implications for This Thesis	12
3	Requirements, Impacts and Constraints	14
3.1	Final Specifications and Requirements	14
3.2	Societal Impact	14
3.3	Environmental Impact	15
3.4	Ethical Issues	15
3.5	Standards	15
3.6	Thesis Management Plan	16
3.7	Risk Analysis	16
3.8	Economic Analysis	16
4	Proposed Methodology	18
4.1	Design Process or Methodology Overview	18
4.1.1	Problem Definition	18
4.1.2	Overall Research Approach	19
4.1.3	Constraints	19
4.2	Preliminary Design / Model Specification	19
4.2.1	Dataset Selection and Handling	19
4.2.2	Data preprocessing	31
4.2.3	Model Design Paradigms and Selection Rationale	33
4.3	Model Architecture Specification	35
4.3.1	Feature representations	35
4.3.2	Dataset split	36
4.3.3	LSTM	37
4.3.4	Autoencoder	39
4.3.5	LSTM Autoencoder	40
4.3.6	Random Forest	41
4.3.7	Evaluation Plan (SIEM-Style)	42
4.4	Implementation of Selected design	45
5	Result Analysis	46
5.1	Experimental Setup	46
5.2	Performance Evaluation	46
5.2.1	Autoencoder	50
5.2.2	Random Forest	53
5.2.3	LSTM	56
5.2.4	LSTM Autoencoder	58
5.3	Final Design Adjustments	62

5.3.1	Autoencoder without validation set balanced, F1-optimized threshold	63
5.3.2	LSTM-AE without validation set balanced, F1-optimized threshold	63
5.3.3	Random Forest with Oversampling	63
5.3.4	LSTM	64
5.4	Statistical Analysis	64
5.5	Comparisons and Relationships	65
5.5.1	Models Detection Quality	66
5.5.2	Operational Feasibility	67
5.5.3	Literature Comparison	67
5.6	Discussions	68
5.6.1	SIEM Deployment Recommendation	68
5.6.2	Alert Fatigue Implications	69
6	Conclusion	70
6.1	Summary of Findings	70
6.2	Contributions to the Field	71
6.3	Limitations	72
6.4	Future Work	73
6.5	Final Remarks	73
	Bibliography	78

List of Figures

3.1	Gantt Chart of Thesis Management	16
4.1	Preprocessing Pipeline	23
4.2	HDFS_v1 Preprocessed Folder Structure	23
4.3	Contents of HDFS.npz.csv	27
4.4	Histogram of normal vs anomaly classes in hdfs.npz y_data	27
4.9	Event Feature Correlation Heatmap	30
4.10	Double-stack LSTM Architecture Used for next-event Prediction adapted from [1]	37
4.11	Autoencoder architecture, adapted from: Weights & Biases, "An Introduction to Adversarial Latent Autoencoders," 2021. [Online]. Available: https://wandb.ai/authors/alae/reports/An-Introduction-to-Adversarial-Latent-	39
4.12	LSTM AE architecture, adapted from Bekkouche et. al [24]	40
4.13	Random Forest Architecture	41
5.1	Confusion Matrix for the Autoencoder Model	50
5.2	Training and Validation Loss for the Autoencoder Model	50
5.3	ROC and Precision-Recall curves for the Autoencoder Model	51
5.4	Confusion Matrix for the Autoencoder Model	51
5.5	Training and Validation Loss Curves for AE	52
5.6	ROC-AUC and PR curve for AE	52
5.7	Confusion Matrix for Random Forest with Undersampling	53
5.8	ROC and Precision-Recall Curves for the RF Model	53
5.9	Random Forest ROC and Precision-Recall Curves (Oversampling)	54
5.10	Random Forest ROC and Precision-Recall Curves (Oversampling)	54
5.11	Random Forest Feature Importance	55
5.12	SHAP Feature Importance of Random Forest Model	55
5.13	Training and Validation Loss Curves for LSTM Model	56
5.14	Confusion Matrix for the DeepLog-Style LSTM Model	57
5.15	ROC and Precision-Recall Curves for LSTM model	57
5.16	ROC and Precision-Recall Curve for the LSTM Autoencoder Model	58

5.17	Training and Validation Loss for the LSTM Autoencoder Model . . .	59
5.18	Confusion Matrix for LSTM-AE, F1-Optimized Threshold	59
5.19	Confusion Matrix for LSTM-AE Model, SIEM-Oriented Threshold . .	60
5.20	Confusion Matrix for LSTM-AE Model, F1-Threshold	61
5.21	Confusion Matrix for LSTM-AE Model, SIEM-Oriented Threshold . .	61
5.22	Training and Validation loss curves for LSTM-AE Model	62
5.23	LSTM-AE ROC and PR curve	62
5.24	Comparison of Key Detection Metrics (Precision, Recall, F1, and FPR) across Evaluated Models	66
5.25	Operational Efficiency Comparison Detailing the Total Training Time (in seconds) and Inference Throughput (in blocks per second) for each model.	67
5.26	Total alert volume generated by each model relative to the ground truth	69

List of Tables

4.1	Comparison Summary of Datasets	22
4.2	HDFS Event Templates	25
4.3	Contents of event_traces.csv	26
4.4	Contents of anomaly_label.csv	26
4.5	Contents of Event_occurrence_matrix.csv	26
4.6	Two Experimental Conditions per Model to Handle Class Imbalance .	33
4.7	Hyperparameters of the LSTM model	38
4.8	Hyperparameters of the Autoencoder Model	40
4.9	Hyperparameters of the LSTM-AE Model	41
4.10	Hyperparameters of the Random Forest Model	42
5.1	Traditional Detection Metrics of All Models on HDFS_v1 Test Set with Condition 1 from Table 4.6	47
5.2	SIEM-Oriented Evaluation Metrics of All Models on HDFS_v1 Test Set with Condition 1 from Table 4.6	48
5.3	Traditional Detection Metrics of All Models on HDFS_v1 Test Set with Condition 2 from Table 4.6	48
5.4	SIEM-Oriented Evaluation Metrics of All Models on HDFS_v1 Test Set with Condition 2 from Table 4.6	49
5.5	Threshold Values of AE and LSTM-AE with Conditions from Table 4.6	63
5.6	Overall Best Results obtained from class imbalance handling conditions	64
5.7	Pairwise McNemar’s Test Results Comparing AE, RF, and LSTM-AE Models	65
5.8	Reported Metrics from Prior HDFS LAD studies	67

List of Equations

Precision	43
Recall	43
F1-score	43
False Positive Rate (FPR)	43
Specificity (TNR)	43
Alert Volume	43
ROC (TPR vs FPR)	43
AUC-ROC	44

Nomenclature

The next list describes several symbols & abbreviation that will be later used within the body of the document

AE Autoencoder

AI Artificial Intelligence

BERT Bidirectional Encoder Representations from Transformers

BGL Blue Gene/L

CNN Convolutional Neural Network

CSV Comma-Separated Values

DL Deep Learning

EDA Exploratory Data Analysis

FN False Negative

FP False Positive

FPR False Positive Rate

GCN Graph Convolutional Network

GNN Graph Neural Network

HDFS Hadoop Distributed File System

LLM Large Language Model

LSTM Long Short-Term Memory

ML Machine Learning

ROC Receiver Operating Characteristic

MSE Mean Squared Error

SIEM Security Information and Event Management

SOC Security Operations Center

TN True Negative

TNR True Negative Rate

TP True Positive

Chapter 1

Introduction

1.1 Background

Presently, computing systems generate huge volumes of system logs which record various things including runtime events, system states, warnings and errors. Logs are required for monitoring system health, diagnosing failures, as well as identifying anomalies or irregular behavior. However, with the scaling of systems in modern times in both size and complexity, it causes a significant increase in log volume. Hence, manual inspection has now become slow and impractical also. Therefore, automated log analysis will be essential for maintaining reliability and supporting security monitoring in the future.

Log anomaly detection concentrate on finding unusual patterns in system logs which show faults in the system, misconfigurations, performance deterioration or security issues. Traditional methods rely on rule-based techniques or classical machine learning models frequently and for this, human interventions are required. It needs manual feature engineering and there are instances where they may struggle to capture complex sequential relationships. On the other hand, recent improvements in deep learning allow stronger modeling of temporal and contextual patterns in log sequences and thus greatly improve anomaly detection performance.

However, SIEM platform centralizes log collection and analysis from multiple sources to support real-time detection and investigation. In such environments, excessive false positives produce alert overload and thus reduce analyst trust and it can cause the limitation of the practical value of anomaly detection systems.

1.2 Rationale of the Study or Motivation

A large portion of log anomaly detection research assesses model performance primarily using metrics which are accuracy-oriented (e.g., accuracy and F1-score) on standardized datasets. Although these metrics are valuable for comparing models, they still may not completely capture operational needs such as reducing alerts generated, maintaining manageable alert volume. In the case of SIEM settings, usability is heavily reliant on false positive behavior due to the fact that repeated false alarms can overload and hence overwhelm analysts and cause alert fatigue.

In addition, recent research and operational reports indicate that a model can have a good performance in benchmark tests and yet be challenging to use in practice if it raises too many alarms. This gap motivates the need of assessment methods that better represent SIEM constraints as well as operational decision-making. Therefore, this study assesses deep learning-based log anomaly detection models using a SIEM-style evaluation perspective while putting emphasis on false positives, alert usability and confusion-matrix-based measures that reflect operational impact.

1.3 Problem Statement

Though deep learning models have been shown to achieve high benchmark scores in the log anomaly detection, the present-day evaluation practice fails to consider the operational issues that define the actual use of the models. In particular:

- Most studies have a tendency to focus on accuracy and F1-score and not to provide enough information about false positive behavior.
- False positive rates are high and reduce the alert utility which leads to fatigue among the analysts in the SIEM setting.
- It is also limited in terms of comparative analyses of deep learning models on the basis of SIEM-relevant criteria, i.e. number of alerts and false alarms.

Therefore, the usefulness of deep learning-based log anomaly detection models with regard to SIEM-style monitoring is not well studied without SIEM-aligned assessment.

1.4 Objective

The objectives of this study are as follows:

- To assess classical machine learning and deep learning methods of detecting log anomalies.
- To evaluate selected models on a published system log dataset using a standardised experimental protocol.
- To evaluate model performance along with the metrics of the confusion-matrix, including the false positive and false negative behaviour.
- To examine the operational usability concerns, such as how alert volume and the practical impact of false alarms.
- To determine the advantages and the weak points of the reviewed models and argue out their appropriateness in the context of SIEM-style implementation.

1.5 Methodology in Brief

The paper compares deep learning-based log anomaly detection models on benchmark system log dataset. The data will be subjected to preprocessing (e.g. turning them into structured event sequences or templates) to produce appropriate model inputs. The chosen models are trained and tested in the same experimental conditions so that fair comparison is achieved.

The metrics used to measure performance are accuracy, precision, recall, F1-score and confusion-matrix-based metrics. SIEM-style analysis is done by looking at the false positives, false negatives, and the nature of alerts to understand how performance on detection can be translated into effective alert utility. These results are used to compare models. It also helps to explain trade-offs between operational noise and detection effectiveness.

1.6 Scopes and Challenges

Scope

This study focuses on:

- Classical ML and DL log anomaly detection evaluation.
- Benchmark system log dataset which are made publicly accessible (e.g., HDFS and BGL).
- Instead of live deployment, SIEM-style metrics of evaluation and usability of alerts.
- Comparative study of performance in detection and operational effects of false alarms under different class imbalance techniques.

This paper neither suggests a new anomaly detector algorithm nor implements an entire SIEM system in practice. Instead, it tests the present methods in the conditions of SIEM-relevant evaluation conditions.

Challenges

Several challenges were considered:

- Anomaly data in system logs is labeled and not always readily available and quite expensive.
- It is common that datasets are imbalanced since anomalous events are rare.
- There are several performance differences that may occur depending on pre-processing decisions and parameters.
- The benchmark findings might not directly be applicable to production SIEM settings due to the difference in data diversity and noise attributes.

Nonetheless, this thesis seeks to bring useful information on SIEM-conformed assessment of deep learning log anomaly detectors.

Chapter 2

Literature Review

This chapter provides reasons why log anomaly detection is an essential part of monitoring modern computing systems and why it matters for SIEM and SOCs, where anomaly predictions translate into alerts and false positives create real workload for analysts. It also contains a summary of the key research lines in the area, ranging from early sequence models, such as DeepLog, to more recent Transformer, graph based, and LLM-based models. It highlights the risks of evaluating models only on offline accuracy, instead of analysing how useful the alerts are in practice.

2.1 Preliminaries

Modern infrastructure, cloud platforms, distributed storage, supercomputers, enterprise networks and others, produce very large amounts of logs each day. These logs are invaluable as evidence to analyze system behavior, diagnose failure and detect anything suspicious. The chapter summarizes the basic notions of log anomaly detection and its connection with SIEM/SOC activities and proceeds with a review of the research results of the early sequence models up to the recent Transformer, GNN and LLM models. It concludes by identifying the main research gaps that provide the motivation for this thesis.

2.1.1 System Logs and Log Anomalies

A system log is a chronological record of software and hardware component-generated events. Each log entry usually consists of a time stamp, level of severity (INFO/WARN/ERROR), name of the component and a text message. Logs are produced in many layers in large-scale systems, application, middleware, operating system, network devices, that is, one incident may appear differently to one observer (or a group of observers).

A log anomaly is an occurrence or a sequence that is not the normal. to a great extent. Practically, an anomaly can be a sign of:

- Crash (failure of the system), time-outs, resources are depleted.
- Performance degradation (unpredictable delays where events are not scheduled differently)
- Security attacks (brute-force, abnormal HTTP traffic and lateral movement)

The use of anomaly detection is necessary since the manual inspection does not scale. He et al. (2016) also stress that real-world log analytics not only includes the process of model selection but also includes a full pipeline to deal with messy data, changing formats, noise and operational constraints [12].

2.1.2 SIEM, SOC and Why ‘Operational’ Evaluation Matters

A SIEM (Security Information and Event Management) system is a system that collects logs and events in a variety of sources, correlates them and provides alerts to be reviewed by an analyst. SOC (Security Operations Centres) are based on the SIEM tool to handle streams of alerts. According to the recent SOC literature, the key issue is not how to identify the anomalies but how to do it in a manner that can guide humans to make efficient decisions.

The authors define alert fatigue: analysts experience a large number of alerts and false positives, losing their trust and slowing response [14]. They suggest doing their solutions by assessing solutions via an Automation, Augmentation, and Collaboration model: automate daily filtering, enhance analysts with better explanations and visualizations and facilitate productive human-AI collaboration [14].

Khayat et al. (2025) describe SIEM as the central nervous system of a SOC and explain how AI/ML can be used to triage, prioritize and detect alerts, with persistent issues such as data quality, false positives and trust/interpretability being recognized [17]. Jalalvand et al. (2024) further state that effective SOC operations not only require proper anomaly detection but also careful alert prioritisation, emphasizing the importance of good quality of incident response on how alerts are ranked and triaged, after detection [26]. This reinforces how anomaly detection goes beyond predictive accuracy, and that the models should be assessed based on their operational usefulness, such as in reducing alert overload.

This thesis takes the stance of the insufficiency of accuracy. A model can be tested with high F1-scores offline but still be invalid of SIEM implementation due to resultant unrealistic alerts or slow processing.

2.1.3 Common Datasets and Benchmark Logs

Public datasets can be used to make reproducible comparisons and a number of studies on log anomalies detection frequently make use of a small set of benchmarks.

To fill a gap in the field of system logs, Zhu et al. (2023) created a comprehensive repository of system logs datasets, covering distributed systems, supercomputers, operating systems, mobile systems and server applications, dubbed as LogHub [20]. Two data sets seem to be used more often:

- HDFS (distributed storage logs, session based organization by block id)
- BGL (Blue Gene/L supercomputer logs, which are usually window-based, as there are no session identifiers)

One of the common results of the literature is that the results may be artificially inflated when they are assessed only using easier datasets or artificial preprocessing. As mentioned by Landauer et al. (2023), there is over-reliance on limited benchmarks and suggests a more realistic evaluation environment and enhanced reproducibility [13]. On the same note, Ali et al. (2025) state that model comparisons should not solely depend on the accuracy but also on the cost of the computation as well as stability to hyperparameter variations [6].

2.2 Review of Existing Research

He et al. (2016) provide an important starting point for log anomaly detection by showing that successful anomaly detection depends on more than just selecting and training a machine learning model. In real operational environments, system logs are often unstructured, noisy, difficult to parse, and dependent on system context. Their study highlights that successful log anomaly detection requires a wider engineering pipeline, including log collection, parsing, grouping, feature extraction, detection, and evaluation. This view is applicable to this thesis as it demonstrates that usefulness of a model should not be judged only in terms of accuracy, but in terms of usefulness also things like computational expense, maintainability, false positive, and reliability in monitoring environments [12].

One of the most important deep learning approaches for log anomaly detection was introduced by Du et al. (2017) through DeepLog. DeepLog uses a Long Short-Term Memory network to model sequences of log events and learn normal execution patterns. If a new log event is not received in the expected sequence based on the learned sequence, If it is not normal behaviour then it is flagged as an anomaly. This work was significant because it introduced sequence learning as a new research strand of log anomaly detection and Many subsequent studies were influenced by this work [1].

Previous and related research reveals log structuring, grouping and representation are necessary steps prior to effective anomaly detection [12], [1], [17]. These studies also demonstrate that log anomaly detection is more difficult when logs are generated by various processes, are noisy, or change following system updates. Meng et al. (2019) further proposed LogAnomaly, a dual-pattern LSTM which incorporates both sequential and quantitative features. They used semantic template embeddings which also addressed the issue of new unseen log templates appearing during deployment. This is useful in real systems, where new or modified log messages often appear over time [27].

CNN-based models are used for learning local patterns in log sequences. A CNN-based detection method was proposed by Lu et al. (2018) which is based on LogKey2Vec embeddings and 1-dimensional convolution on sequences of events. Their results on the HDFS dataset showed that CNNs can learn useful local relationships between nearby log events [8]. Aziz and Munir (2024) discuss deep learning models for log anomaly detection and report that hybrid designs, such as CNN with LSTM, can combine local feature extraction with temporal dependency learning. CNN layers can detect short-range event patterns and LSTM layers can learn long-term sequence behaviour. However, most of these studies primarily focus on testing models offline, and do not consider operational issues like false positive burden or alert volume [10].

Kavlak and Sezen (2026) have done a comparative analysis between Autoencoder, CNN, LSTM and Transformer, using the HDFS dataset. Under identical preprocessing conditions, they found the sequential models to significantly outperform the reconstruction-based approaches – Transformer achieved highest F1-score of 0.69, while Autoencoder achieved lowest of 0.408, due to reconstruction error failing to detect rare anomalies [16]. Also the case with these studies is that they have reported standard ML metrics but not operational ones, which highlights the importance of this thesis to do a SIEM-style evaluation.

Transformer-based models are one important direction in log anomaly detection as they can learn broader context from a log sequence, rather than just reading events one step at a time like many recurrent models. Huang et al. (2020) introduced HitAnomaly, a hierarchical Transformer model that considers both log templates and parameter values [3]. Guo et al. (2021) proposed LogBERT, which applies a BERT-style model to learn contextual patterns in log sequences [2]. Chen and Liao (2022) further continued this direction by introducing BERT-Log, which views like natural language sequences, it can be used to log sequences using a pre-trained BERT to model the semantic and positional information. The model was tested on Both HDFS and BGL reported good F1-scores for both the datasets. This is important because BGL has more complex and diverse log relationships than HDFS, useful for testing if a semantic model can deal with more difficult log data. But despite the potential of BERT-based models that these results demonstrate, there are still challenges to be addressed. For these models, they are primarily reported in terms of benchmark performance [29]. In addition to this, for SIEM-style use, false positives, alert volume, robustness and usefulness of the model output for analysts must be considered.

A major challenge in log anomaly detection is the preprocessing pipeline, especially log parsing. Log parsing converts raw log messages into structured templates, but this process can be computationally expensive, and sometimes inaccurate. Errors in parsing may also affect the performance of anomaly detection models. To address this problem, Le and Zhang (2022) proposed NeuralLog, a parsing-free Transformer-based model that works directly with raw log messages [21]. However, reducing parsing does not mean that preprocessing becomes unnecessary. Even parsing-free methods need careful grouping, windowing, encoding and evaluation design [7],[13].

Many existing models focus mainly on sequence anomalies, where abnormal behaviour appears as a change in the order of log events. Li et al. (2020) proposed SwissLog to address this issue by combining BERT-based semantic embeddings, time embeddings, and an attention-based Bi-LSTM model. They tested their approach on various LogHub datasets such as HDFS and BGL and it was aimed to identify sequence anomalies as well as performance anomalies [18]. To deal with unstable log data, Zhang et al. (2019) proposed LogRobust. They claim that models based on a large number of fixed log template IDs can be broken when software update adds or changes templates [28]. The main takeaway from these studies is that effective log anomaly detection should consider not only what event occurred, but also when it occurred, what values it contained, and whether the model can remain reliable when log formats evolve [18], [28]. This is particularly critical for

environments like SIEM, where logs can be from various systems and can evolve over time.

Graph-based models offer a more flexible way to model relationships between log events. Alsalmi et al. (2025) proposed a BertGCN-based approach that combines BERT embeddings with Graph Convolutional Networks.[9].

In recent times, large language models have also been introduced into log anomaly detection. Guan et al. (2025) proposed LogLLM, that is a Transformer encoder (BERT) with an LLM decoder (Llama) used to detect anomalies in a variety of datasets, such as HDFS and BGL. However, LLM-based methods introduce new practical concerns, including computational cost, memory requirements, and inference latency [15]. Himler et al. (2024) consider federated deep learning on log-event sequences to reveal issues such as preprocessing traps (e.g., the unnaturally inflated result due to padding), concept drift, imbalance, and implementation limitations [11]. Their effort supports the need to have realistic and deployment-conscious methodologies of evaluation.

Unsupervised approaches are also important because labelled anomalies are often difficult to obtain in real systems. Farzad and Gulliver (2020) proposed an unsupervised approach of combining two Autoencoders with one Isolation Forest, trained without labels. They achieved accuracy of above 99% on datasets like BGL, Openstack and Thunderbird. Their model showed a robust and computationally faster solution for large-scale system log anomaly detection through the Autoencoder’s feature extraction and the Isolation Forest’s ability to isolate normal data [30].

Recent literature has also raised doubts on the use of complex deep learning models are always necessary. Yu et al. (2024) empirically evaluated and contrasted deep learning and classical machine learning approaches, and concluded that the results depend significantly on the preprocessing, dataset characteristics, and evaluation design. Ali et al. (2025) further demonstrated that traditional supervised machine learning approaches, particularly Random Forest, can be as effective as deep learning models with the advantage of reduced training time and less hyperparameter tuning. The results indicate that the practical usefulness is not necessarily related to the complexity of the model [4], [6].

Albert (2025) also suggests that the researchers might be more concerned with the benchmark scores and less with practical challenges like noise, concept drift, and operational constraints. This is a very pertinent criticism as a model with high F1

score on a benchmark dataset may not be appropriate for deployment if it generates too many false positives or fails to perform when the log conditions change. Operational factors, such as pace, stability, false positivity, and performance in realistic conditions, should be strictly evaluated [5].

Anomaly detection is also closely related to SIEM, as SIEM systems are used to collect, correlate and alert on large amounts of security and system logs. There are several recent works that relate to anomaly detection, SOC and SIEM challenges. The issue of alert fatigue is discussed by Tariq et al. (2025) and Khayat et al. (2025), who also highlight the importance of metrics beyond just detection accuracy. Too many false positives can overload SOC analysts and diminish the value of the detection system in a SOC environment. Thus, apart from benchmark accuracy, other factors to consider when integrating SIEM are alert volume, false positive rate, false negative behaviour and analyst usability [14], [17].

In general, the literature reveals that log anomaly detection has evolved from traditional feature-based approaches to LSTM, CNN, Transformer, graph-based, federated, unsupervised, and LLM-based approaches. But many of the current efforts are still primarily directed towards offline benchmark performance. Not as much focus is placed on the behavior of these models when the output is used as an alert in a SIEM style monitoring environment. This thesis fills that gap by examining deep learning-based log anomaly detection both from the perspective of standard deep learning metrics (accuracy, precision, recall, F1-score) and SIEM-relevant metrics (false positive behaviour, false negative behaviour, and alert volume).

2.3 Summary of Key Findings

2.3.1 Key Results and Trends

From the reviewed studies , several important trends can be seen:

1.Sequence modeling became the default framing.

Firstly, sequence modelling has become one of the main approaches in log anomaly detection. However, DeepLog revealed that many of the anomalies are not only contained in a single log line, but in the sequence of log events [1]. This was later expanded to learn more complex relationships between the logs in the sequence by using later models such as LSTM-based models and Transformer-based models [2], [3].

2. Performance is highly sensitive to the representation selections.

Second, the representation of logs in the model affects the model performance strongly. Some rely on parsing logs but this can lose information, or be unreliable if the log template changes [7]. Due to this, recent works on parsing-free approaches, semantic embeddings and hybrid representations have been investigated to create more robust models [18], [9].

3. Benchmarks are useful but can be misleading if they are used alone.

Thirdly, benchmark datasets like HDFS and BGL are helpful to compare models but are not comprehensive of real deployment conditions. Some studies and surveys indicate that the high benchmark scores do not necessarily imply that a model is ready for practical use [5] and [13]. Hence, there is more emphasis on the fair evaluation, repeatability and realistic test conditions in recent studies [6], [20].

4. Operational usefulness is becoming central (SIEM/SOC angle).

Finally, useful in operation is becoming more and more important. In SOC/SIEM setting, the model identifies anomalies and generate alerts. And those alerts can be managed realistically by analysts. Too many false alerts can cause alert fatigue if the analysts become overwhelmed [14]. Therefore, the prioritization of alerts and deployment in real security monitoring systems are also the focus of recent research [17], [26].

2.3.2 Implications for This Thesis

Based on the literature reviewed, it is concluded that detecting logs should not be considered solely as a machine learning classification problem. In real-world cybersecurity scenarios, models should also be assessed for their usefulness and manageability within SIEM-type monitoring systems. There are three gaps that combine to motivate this thesis:

- **Gap 1: Benchmark accuracy vs operational usefulness.**

While many studies show high accuracy or F1-scores, practical aspects such as false positives, number of alerts, and workload of analysts are not often discussed [5], [14]. In real SOC environments, these factors are important because large numbers of unnecessary alerts can reduce system usability and increase alert fatigue.

- **Gap 2: Preprocessing and evaluation can strongly influence results.**

Preprocessing can be explicitly controlled and justified, as padding, grouping and windowing decisions can result in information leakage or unrealistic

advantages [11], [13].

- **Gap 3: Classical vs Deep learning models (comparative studies) under constant conditions is restricted.**

Recent studies suggest Classical ML models prove to be effective on log datasets, competing with DL models [4], [6]. However, direct comparisons under identical preprocessing and evaluation conditions is limited, hence, our initiative to compare some well-known models under the same experimental protocol.

- **Gap 4: Class imbalance handling is not empirically justified in most studies.**

Most studies do not systematically compare the superiority of one class imbalancing technique over another, with adequate experimentations, which makes it difficult to choose techniques according to the model/paradigm, hence we compare some class imbalance methods under identical test conditions, assessing the impact on SIEM-oriented metrics.

Therefore, this thesis adopts the following research focus which is to evaluate representative log anomaly detection models on standard log dataset (HDFS dataset from LogHub) under a SIEM-style evaluation lens, emphasizing both detection quality and operational practicality [22], [14].

Chapter 3

Requirements, Impacts and Constraints

This chapter discusses the main requirements, impacts, and constraints of the thesis. It explains the technical needs of the study, including dataset, models, evaluation metrics, software, and hardware. It also presents the societal, environmental, ethical, economic, and management-related aspects of the thesis.

3.1 Final Specifications and Requirements

To assess the deep learning-based log anomaly detection in a SIEM-like environment, some technical and methodological aspects were set. A publicly available system log dataset is needed for the study, which can be used for sequence-based and numerical analysis. Therefore, the preprocessed HDFS v1 data set was chosen. The required files include Event_traces.csv, HDFS.log_templates.csv, anomaly_label.csv, Event_occurrence_matrix.csv, and HDFS.npz.

3.2 Societal Impact

This research makes a contribution towards enhancing awareness on cybersecurity practice because it emphasizes the possibility of abnormal system behaviour being reflected through recurring log patterns. Examining log anomalies in system datasets helps towards improved insights into software reliability, system monitoring, and security risks in digital infrastructure. The results can be used by developers, system administrators, organizations, and researchers to improve automated log monitoring practices and enhance the overall safety of computing systems. The study developed anomaly detection models in the context of Security Operations Centers (SOCs). One advantage of using the models is that manual log inspections

can be decreased. Security analysts can then dedicate their time and effort to the evaluations of abnormal behavior of the systems.

3.3 Environmental Impact

This study's contribution to the environmental footprint is mostly in the computational costs. Power is consumed when deep learning systems are both trained and tested. There is not a lot of size to the models that are selected for the study, but if the experiments are performed a large number of times, costly waste of energy will likely occur. The study's goal is to employ a limited number of experiments using models of a reasonable size to limit the environmental footprint. Captured settings of the models, logs, and results are utilized so that previously performed experiments are not executed again. The study also avoids redoing heavy preprocessing of the data. The experiments are constrained in their computational costs, and therefore, use reasonable sized models and open-source resources. Thus, the environmental footprint of the study is rather limited.

3.4 Ethical Issues

The study follows standard academic and data ethics. The HDFS data repository is a preprocessed public data repository, and there are no privately owned security data logs and no private user data. The study also aimed to not target users, systems of any individual or organization. This study conducts analyses of logs and the detection of data level anomalies. The results of analyses are for academic and analytical purposes to promote and refine log anomaly detection and monitoring of cybersecurity.

3.5 Standards

This work concerns the existing practices in cybersecurity and logging. NIST SP 800-92 outlines the needs and practices for the collection and analysis of security logs. The NIST Cybersecurity Framework 2.0 recognizes the necessity of identifying unusual behaviors within systems. The study further conforms to the monitoring and risk assessment practices covered in ISO/IEC 27001. To enhance the quality of research, the same dataset split, data preprocessing, model configurations, and metrics for model evaluation were used in all research cases.

3.6 Thesis Management Plan

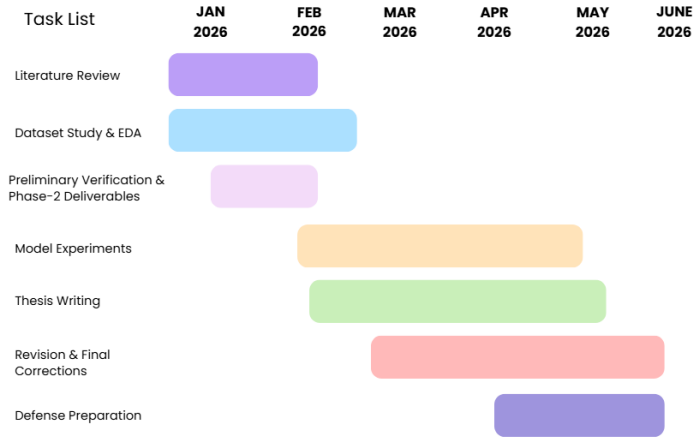


Figure 3.1: Gantt Chart of Thesis Management

Fig 3.1 shows the major phases of the thesis from literature review to defense preparation. The thesis followed a structured timeline which started from literature review and dataset analysis. Preprocessing and exploratory data analysis were done to examine the class imbalance, event frequency, sequence length, and the distribution of anomaly. Preliminary verification, model development, and experimentation were conducted during the middle stages, followed by thesis writing, revision, and defense preparation before final submission.

3.7 Risk Analysis

This research considered a variety of risks. Risk with data was minimized by choosing a benchmark dataset that was validated and made publicly available. Technical risks, such as overfitting, underfitting, class imbalance, and incorrect threshold, were managed by validation and training-loss, along with a confusion matrix. Another risk was that log anomaly datasets tend to be imbalanced, thus high accuracy could be a misleading result. This risk was managed by reporting precision, recall, F1-score, false positive rate, PR-AUC, alert volume. The risk of evaluation being too optimistic was managed by using block-level evaluation so that inflation of alert counts would not occur.

3.8 Economic Analysis

This research was performed at a low financial cost, as it relied on publicly available dataset and used open source software. The only direct cost was approximate

\$9.99 per month spent for a subscription on Google Colab to train and test the models. There were no special hardware costs, paid datasets, or enterprise software costs. The project was affordable when compared to commercial SIEM systems and cloud security monitoring services. The study illustrates log anomaly detection can be evaluated based on false alerts, missed anomalies, and on the value of monitoring.

Chapter 4

Proposed Methodology

Comparison of mean event frequencies, as shown in Figure 4.7 below, between normal and anomalous blocks reveal clear differences that justify the use of frequency-based models. Some events never appear in normal blocks, such as E7, E8, E10, E12, E13, E14, E15, and E29, indicating that any occurrence of these events in a block is a strong anomaly signal. Core execution events E9, E11, and E26, appear consistently in almost all normal blocks but drop significantly in anomalous blocks, indicating that the absence of those events signals anomaly. Events such as E20, E18, (error related events) increase in occurrence substantially in anomalous blocks.

4.1 Design Process or Methodology Overview

This chapter explains the complete methodology plan, which includes the full experimentation pipeline, from dataset selection to model selection and SIEM-based evaluation. The pipeline consists of selecting the HDFS_v1 benchmark dataset, selecting the appropriate models for the comparisons, representing the logs in suitable ways for model training and evaluation criteria, and, finally, implementation and evaluation of the pipeline against SIEM-style metrics.

4.1.1 Problem Definition

To lessen the burden of SOC analysts, there is an increased need for ML/DL to be incorporated for faster anomaly detection and reliable alert generation from SIEM systems [14]. SOC analysts face alert fatigue when overburdened with too many anomaly alerts from SIEM systems, making it harder for them to focus on genuine threats [14]. Deep learning, with its non-linear nature, offers an impressive way to improve SIEM anomaly detection, as it automatically learns patterns in logs instead of having to learn manual rules. Models such as LSTMs, Autoencoders, and their hybrid variants can capture long-range relationships and context, hence can detect

unusual event sequences. SwissLog and LogLLM have demonstrated that DL can adapt to varying log formats and sizes for bigger datasets [18],[15]. DL, with proper thresholding and hyperparameters, has the ability to reduce False Positive Rates by learning patterns better.

4.1.2 Overall Research Approach

This thesis follows a comparative, research-based methodology to evaluate deep learning approaches for log anomaly detection using a SIEM-style evaluation lens. The research pipeline is divided into stages: log collection, preprocessing, model training, and evaluation. Extensive exploratory data analysis is done to truly understand the nature of the dataset. Comparative evaluation shows class imbalance handling techniques trade-offs, uses confusion matrix metrics such as Precision, Recall, and F1 score to measure model performance, while SIEM-style operational metrics are calculated, such as false positive rate and alert volume, that reflect practical SOC concerns. T

4.1.3 Constraints

Computational resources: Hardware limitations require careful strategies; for example, preprocessing large CSV files requires significant CPU time. Extensive GPU access is required to train deep learning models such as LSTM. Proper selection of hyperparameters is essential to optimize training with limited GPU access. Limited disk space may result in out-of-memory errors. The study is further constrained to a single benchmark dataset, HDFS.v1, due to time and resource limitations. The results may not generalise to other log datasets or production SIEM environments.

4.2 Preliminary Design / Model Specification

4.2.1 Dataset Selection and Handling

We have chosen the Hadoop Distributed File System (version 1) dataset as our primary dataset. It is available in the LogHub [20] GitHub repository and is downloaded from Zenodo, which hosts the LogHub datasets. In many log anomaly detection studies, HDFS and BGL have been consistently held as the canonical datasets, but LogHub [20], the repository containing the 19 suitable datasets for log anomaly studies, reports that HDFS is one of the most downloaded and widely used datasets. The raw logs are records of system-level events produced by the Hadoop Distributed File System. Logs were obtained during normal and faulty operations. The original dataset has logs spanning 38.7 hours, containing over 11 million log lines, with size

1.47 GB in raw form. It was first introduced in DeepLog [1], where logs were parsed into traces based on block IDs and ground truth labels assigned as normal or anomalous. Foundational studies such as LogBERT, HitAnomaly were based on HDFS [2], [3]. Transformer-based models from those studies reported strong performance on this dataset, primarily due to its stable templates and consistent labels. SwissLog [18] reported a very high F1-score, even though they simulated drift on the logs.

For LAD, HDFS_v1 stands out as the most suitable dataset choice for our work, compared to other versions of HDFS such as HDFS_v2, HDFS_v3 and the BGL dataset. The main advantage of HDFS_v1 is that the logs are grouped into particular operation blocks based on unique block IDs, which reflects specific program execution paths, and are all assigned explicit binary ground truth labels (normal or anomalous), meaning every execution trace of the Hadoop distributed file system is clearly marked. This labelling has been done by experts using handcrafted rules for LAD, giving researchers a reliable benchmark to measure accuracy, F1 score, precision and recall. Whether the model is unsupervised or supervised, the data can be used for training and then labels can be used for evaluation to determine how well the model performs.

Another version, HDFS_v2, has no labels at all. Although it is useful for unsupervised tasks like log parsing, it cannot be used to quantitatively evaluate LAD models using standard detection metrics, since the lack of ground truth means the evaluation metrics cannot be calculated; there is no way to know if the predicted anomalies are correct.

HDFS_v3 is also labeled but has synthetic anomalies generated through fault injection experiments, hence the data cannot be treated as natural system logs. The injected anomalies are often obvious and follow repetitive patterns, making it easier for models to show high detection performance, but real-world anomalies are unpredictable in nature — mixed with normal behaviour, often partial failures, and do not cleanly separate in feature space. Artificial fault injected anomalies often follow fixed, repetitive patterns, making it easier to detect those hence resulting in inflated benchmark scores.

On the other hand, BlueGene/L (BGL) supercomputer logs are message-wise labeled rather than blockwise, and this lack of blockwise grouping requires researchers to use time intervals or session windows, which creates noisy data and makes it difficult to capture actual execution semantics of the system [27]. Modern SIEMs usually analyse entire execution sessions to raise alerts for any abnormal system behaviour,

not logs line by line. HDFS_v1 avoids this issue by sequencing logs naturally into traces by matching block IDs, which is well suited for sequential-based deep learning models prominent for LAD research. Moreover, HDFS_v1 has only 2.93% anomalies whereas BGL has varying and greater portions of anomalies depending on grouping. The small proportion of anomalies allows models to be less biased towards the majority class. Additionally, HDFS_v1 is smaller in size than BGL, making it more manageable for researchers with limited resources.

Dataset	HDFS_v1 (Selected)	HDFS_v2	HDFS_v3	BGL
Size	1.47 GB	16.06 GB	2.96 GB	708.76 MB
#Lines	11,175,629	71,118,073	14,778,079	4,747,963
Year Created	2009	circa 2016–2018	2020	2007
Log Grouping	Block-level using block_id (natural execution traces). Example: all logs for one file block	Node-level logs (no execution sessions). Example: mixed logs from a machine	Synthetic fault injected anomalies. Ex: artificial failure traces	MMessage level logs(not belonging to natural sessions). Ex; single error message
Label Availability	Fully labeled (normal/anomaly)	Unlabeled	Labeled	Message-wise labels only
Nature of Anomalies	Natural system faults mixed with normal behaviour. Example: short or incomplete execution	Unspecified	Fault-injection anomalies (synthetic, repetitive patterns). Example: injected disk failure	Real failures split across individual log messages
Suitability for Quantitative Evaluation	Standard metrics usable (Precision, Recall, F1-score, FPR)	No ground truth; metrics not computable	Metrics can be calculated but may be inflated	Limited; depends on artificial grouping strategy
Practical Limitations	Only 4.7% of sequences are unique, potentially inflating scores	No quantitative evaluation possible	Not representative of real-world failures	Unstable anomaly ratio due to different windowing methods
References	[20], [23]	[20], [23]	[20], [23]	[20], [25]

Table 4.1: Comparison Summary of Datasets

Thus, HDFS_v1 is an ideal choice for this study. It is preprocessed and manageable in size, comes with labels, and has realistic anomalies, thus aligning with our time and resource constraints. From traditional ML to advanced DL architectures for log

anomaly detection, HDFS has proven to be a gold standard.

This HDFS_v1 dataset is suitable for all four models evaluated in this study, since its block-level frequency vectors and sequences of events are ready to be directly used as fixed-length input for both classical and deep learning classifiers, without needing extra preprocessing steps.

Exploratory Data Analysis:

Prior to the experiments, an exploratory data analysis was done to understand the dataset, analysing the file structures, data types, class distribution, sequence characteristics, event frequency patterns. The findings determined preprocessing decisions and model selection rationale.

Dataset File structure:

HDFS_v1 is already a preprocessed and structured version of the original Hadoop Distributed File System logs [20],[23]. Unlike raw system logs, HDFS_v1 is already parsed, organized by the creators in the following ways: log templates extracted from raw textual log messages, each template mapped to a unique Event_ID, log events grouped into traces based on block_IDs, binary labels assigned for each block.

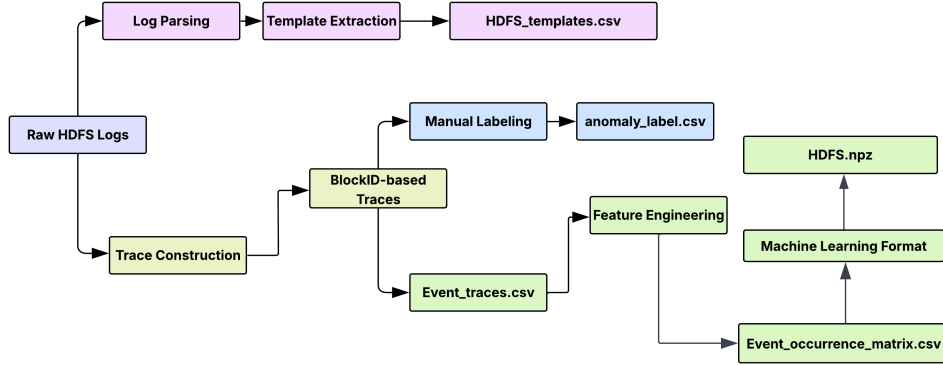


Figure 4.1: Preprocessing Pipeline

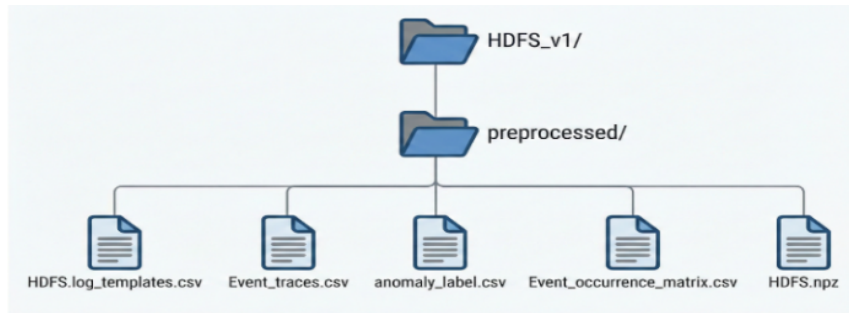


Figure 4.2: HDFS_v1 Preprocessed Folder Structure

HDFS.log_templates.csv: Contains log templates extracted from raw log messages after parsing. Each template represents a type of system event. Each template has a unique Event_ID. Contents are as follows:

All 29 event templates:

Table 4.2: HDFS Event Templates

EventID	Template
E1	[*]Adding an already existing block[*]
E2	[*]Verification succeeded for[*]
E3	[*]Served block[*]to[*]
E4	[*]Got exception while serving[*]to[*]
E5	[*]Receiving block[*]src:[*]dest:[*]
E6	[*]Received block[*]src:[*]dest:[*]of size[*]
E7	[*]writeBlock[*]received exception[*]
E8	[*]PacketResponder[*]for block[*]Interrupted[*]
E9	[*]Received block[*]of size[*]from[*]
E10	[*]PacketResponder[*]Exception[*]
E11	[*]PacketResponder[*]for block[*]terminating[*]
E12	[*]:Exception writing block[*]to mirror[*]
E13	[*]Receiving empty packet for block[*]
E14	[*]Exception in receiveBlock for block[*]
E15	[*]Changing block file offset of block[*]from[*]to[*] meta file offset to[*]
E16	[*]:Transmitted block[*]to[*]
E17	[*]:Failed to transfer[*]to[*]got[*]
E18	[*]Starting thread to transfer block[*]to[*]
E19	[*]Reopen Block[*]
E20	[*]Unexpected error trying to delete block[*]BlockInfo not found in volumeMap[*]
E21	[*]Deleting block[*]file[*]
E22	[*]BLOCK* NameSystem[*]allocateBlock:[*]
E23	[*]BLOCK* NameSystem[*]delete:[*]is added to invalid-Set of[*]
E24	[*]BLOCK* Removing block[*]from neededReplications as it does not belong to any file[*]
E25	[*]BLOCK* ask[*]to replicate[*]to[*]
E26	[*]BLOCK* NameSystem[*]addStoredBlock: blockMap updated:[*]is added to[*]size[*]
E27	[*]BLOCK* NameSystem[*]addStoredBlock: Redundant addStoredBlock request received for[*]on[*]size[*]
E28	[*]BLOCK* NameSystem[*]addStoredBlock: addStoredBlock request received for[*]on[*]size[*] But it does not belong to any file[*]
E29	[*]PendingReplicationMonitor timed out block[*]

Event_traces.csv: Stores sequences of log event IDs, grouped by block IDs. Each row has columns: $block_ID$, $label(success/fail)$, $type$, $features$, $timeinterval$, and $latency$. Since several

Block_ID	Label	Type	Features	TimeInterval	Latency
blk_160899...	Success	-	[E5,E22...]	[0.0,1.0,..]	3802
blk_750348...	Success	-	[E5,E5,E22..]	[0.0,0.0,..]	3802
blk_354458...	Fail	21	[E5,E22,..]	[0.0,1.0,..]	3797
blk_907399...	Success	-	[E5,E5,..]	[0.0,1.0,..]	50448

Table 4.3: Contents of event_traces.csv

anomaly_label.csv: Provides the corresponding binary labels (normal/anomaly) for each unique trace//block.ID. Contents are as follows:

Block_ID	Label
blk_1608999687919862906	Normal
blk_7503483334202473044	Normal
blk_3544583377289625738	Anomaly
blk_9073992586687739851	Normal

Table 4.4: Contents of anomaly_label.csv

Event_occurrence_matrix.csv: Represents traces as fixed-length vectors based on event occurrence counts. Each row consists of columns: $block_ID$, $label(success/fail)$, $type$, E1 through E29, that store counts of each event occurrence within the block. Contents are as follows:

Block_ID	Label	Type	E1	E2	E3	E4	E5	E6	E7
blk_160899...	Success	-	0	0	203	0	10	7	0
blk_750348...	Success	-	0	2	1	0	3	0	0
blk_354458...	Success	21	0	0	203	0	3	0	0
blk_907399...	Success	-	0	3	0	0	3	0	0

Table 4.5: Contents of Event_occurrence_matrix.csv

HDFS.npz: A ZIP-compressed NumPy archive that has integrated the event occurrence vectors with the corresponding block-level ground truth labels. This numerical vectorized input is ready to be directly fed into ML/DL models. Contents are as follows:

```

one example of x_data from HDFS.npz:
[list(['E5', 'E5', 'E5', 'E22', 'E11', 'E9', 'E11', 'E11', 'E9', 'E9', 'E26', 'E26', 'E26'])]
Data type of x_data: object

one example of y_data from HDFS.npz:
[0]
Data type of y_data: int64

```

Figure 4.3: Contents of HDFS.npz.csv

Figure 4.3 below shows the amount of normal vs anomalous ground truth labels in y_data:

Class distribution: The dataset contains 575,061 blocks in total. 558,223 are normal (97.07%) and 16,838 are anomalous (2.93%), a very skewed ratio of approximately 33:1, confirming the severe class imbalance documented in prior work [22]. As shown below in Figure 4.4, the imbalance is huge. A regular classifier predicting all blocks as normal would still obtain 97.07% accuracy, while detecting zero anomalies, reinforcing that simple accuracy alone is an insufficient evaluation metric for this dataset.

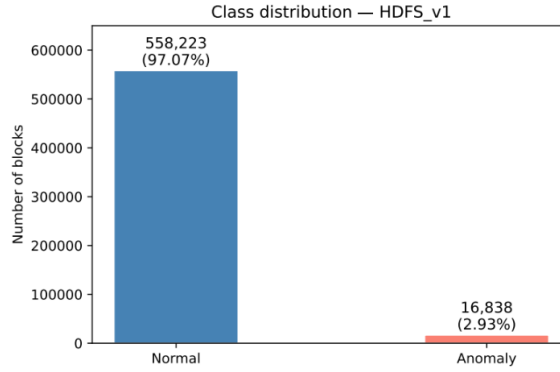


Figure 4.4: Histogram of normal vs anomaly classes in hdf5.npz y_data

Therefore, every data instance of the HDFS_v1 is a sequence of event IDs corresponding to a single unique block, representing the block’s execution trace, and the block has a manually expert-assigned ground truth label (normal/anomalous). This well-maintained structure diminishes the need to do log parsing or template mining for our work.

Sequence length analysis: Analysis of block trace lengths showed that normal and anomalous blocks have very different length distributions, as shown below in Figure 4.6. Normal blocks show quite consistent lengths with mean of 19.5 events, standard deviation of 4.8, and 75% of blocks containing between 19 and 20 events. Anomalous blocks have more variations, with a mean of 17.1 events, standard deviation of 12.4, and 25% of blocks with only 4 events. The minimum length for

normal blocks is 13 events, and 2 for anomalous blocks. This suggests that incomplete or prematurely terminated execution traces are a strong indicator of system faults. This finding directly justifies selecting window size 20 for sequential models, as it covers the execution length of the vast majority of normal blocks. Also highlighted is a limitation of the LSTM model, i.e, blocks shorter than the window size are invisible to the model during evaluation as those produce no sliding windows.

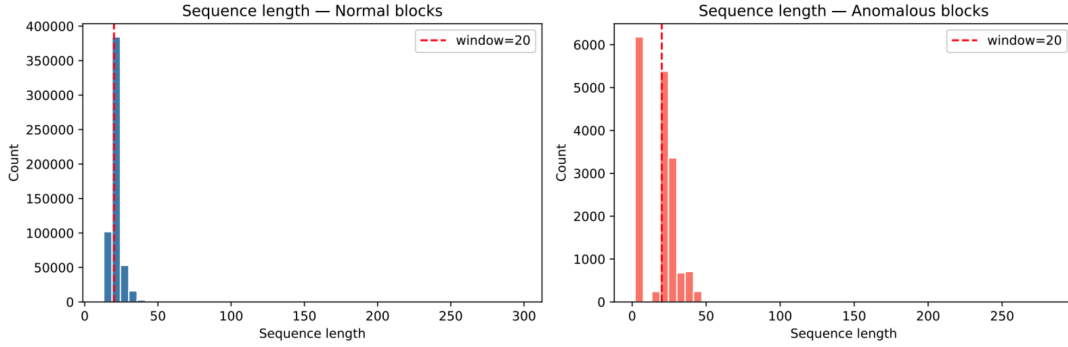


Figure 4.5: Sequence Length Distribution by Label

Event frequency analysis:Figure 4.6 below shows the top 15 most frequently occurring events across all blocks in the dataset. The Events E5, E26, E11, E9, E21, and E23 dominate, which reflects the core normal execution pipeline of HDFS. E22 appears exactly 575,061 times, meaning once in every block, making it a constant feature that carries no distinguishable information.

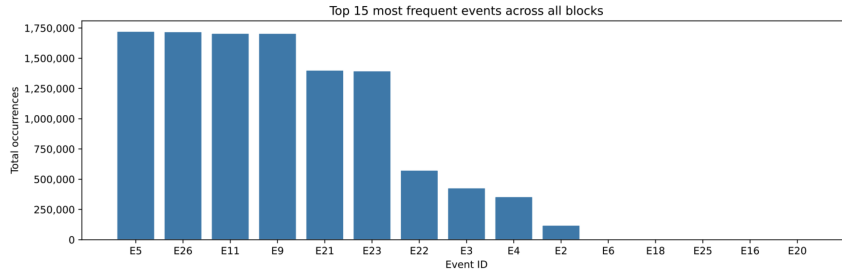


Figure 4.6: Top 15 Most Frequent Event Occurrences Across All Blocks

Comparison of mean event frequencies, as shown in Figure 4.7 below, between normal and anomalous blocks reveal clear differences that justify the use of frequency-based models. Some events never appear in normal blocks, such as E7, E8, E10, E12, E13, E14, E15, and E29, indicating that any occurrence of these events in a block is a strong anomaly signal. Core execution events E9, E11, and E26, appear consistently in almost all normal blocks but drop significantly in anomalous blocks, indicating that the absence of those events signals anomaly. Events such as E20, E18, (error related events) increase in occurrence substantially in anomalous blocks.

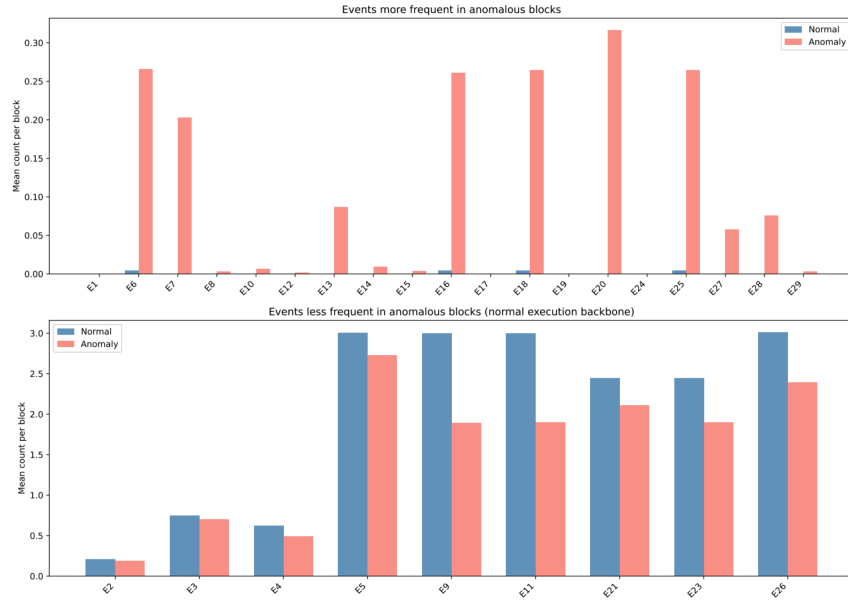


Figure 4.7: Mean Event Frequencies — Normal vs Anomalous blocks

We applied **Jensen-Shannon Distance** to each event, to examine how those events differ in frequencies between normal and anomalous blocks. JSD measures how different two probability distributions are, measuring the level of dissimilarity between patterns. A higher value indicates higher dissimilarity. Figure 4.8 below shows 15 out of 29 event types showed JSD above the 0.1 threshold, confirming that more than half the events appear in different frequencies between normal and anomalous blocks. E26 and E5 showed the highest JSD values, 0.546 and 0.502 respectively. E22 showed a JSD of exactly 0.0, since it is a constant feature appearing once in every single HDFS block, holding no distinguishable effect. The mean JSD is 0.166, hence a strong justification for frequency-based anomaly detection on this dataset. Since it is calculated from event frequencies rather than event order, it specifically captures divergence in the count of events' occurrences. Therefore, this directly justifies any frequency-based anomaly detection model on the dataset due to showing clear frequency distinctions between the two classes.

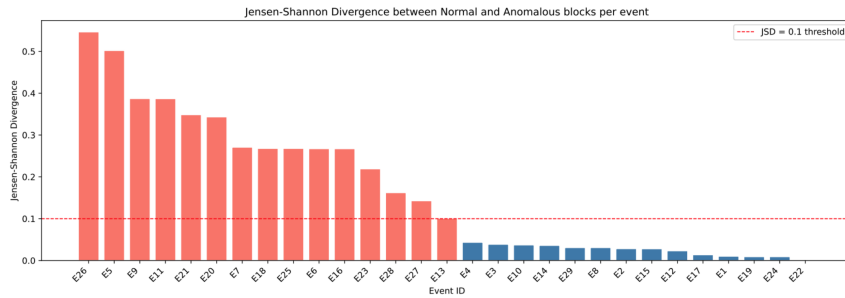


Figure 4.8: Jensen-Shannon Distance per Event

Event Correlation Analysis: A Pearson correlation analysis is done. Generating

a correlation heatmap of the 29 event features revealed that events cluster into groups with similar co-occurrence patterns, as shown below in Figure 4.10. Events E5, E9, E11, and E26 form a strongly correlated cluster because these are the core normal execution events that appear together in every normal block and are absent when a fault occurs. E22 shows near-zero correlation with all other events, consistent with its JSD of 0.0 signifying its role as a feature that constantly appears for all blocks.

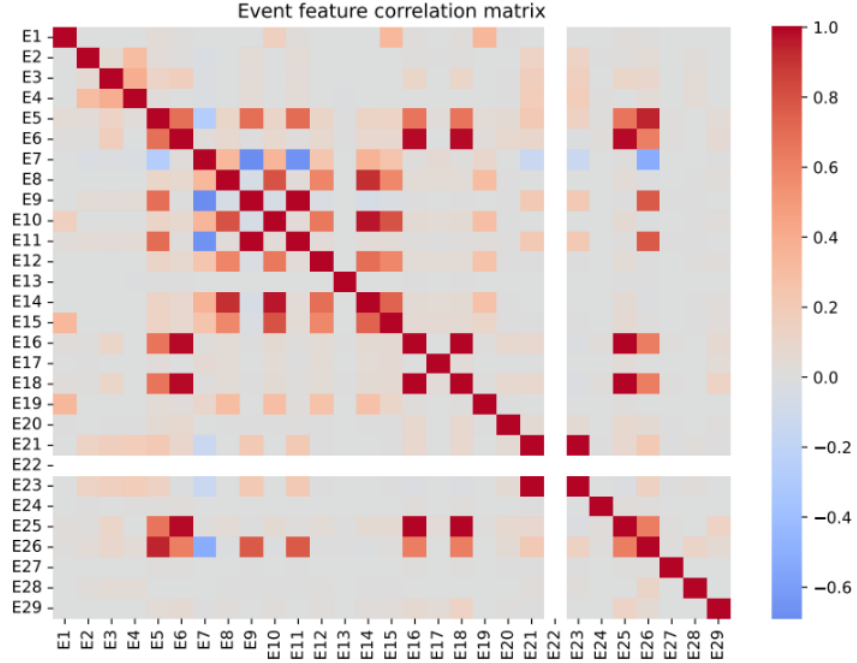


Figure 4.9: Event Feature Correlation Heatmap

Dataset limitations:

HDFS_v1 is highly imbalanced. Anomalous data makes up 2.93% of the entire block-wise data [22]. Such skewed data may cause unfair bias for models towards normal data, which leads to high accuracy but poor recall. Hence, it is important to evaluate models on F1-score, Precision, Recall, as well as False Positive Rate (FPR) and True Negative Rate (TNR). Careful preprocessing is required to achieve optimum performance with this constraint.

Class imbalance is a well-documented challenge in log anomaly detection research. HDFS_v1 exhibits a skew that reflects realistic operational conditions with only 2.93% of blocks labelled as anomalous. Real conditions where genuine faults are rare relative to the volume of normal system activity [22]. The imbalance affects model training differently. How each model in this study addresses this challenge is detailed in Section 4.2.2.

The EDA clearly highlights the events’ occurrence counts deviation between the classes. This characteristic cemented the need to investigate reconstruction, frequency-based models more for this dataset, which we do not see much of from the previous studies, more of which focus highly on sequential representations.

4.2.2 Data preprocessing

The pre-processed curated files folder from HDFS.v1 is used directly. The pre-processing pipeline standardized in [20], is adopted in this study. Log sequences grouped block-wise prevent traces from leaking across dataset splits. The dataset is split 70/15/15 for training, validation, and test sets respectively, consistent with the methodological requirements specified for this study and aligned with established practice in log anomaly detection benchmarking [6]. A fixed `random_state` of 42 is used [4].

For reconstruction-based models, anomaly detection thresholds are determined after training using reconstruction errors from the validation set, with the threshold set at the F1-optimized or SIEM-optimized(maximizing recall, keeping false positive rate as low as possible). No artificial anomaly is injected, and no class resampling is applied to the deep learning models, keeping the focus on model behaviour rather than data manipulation.

A window size of 20 is applied for all sequential models, to match HDFS block traces mean length 19.43 events per block, making a window of 20 sufficient to capture a full execution sequence without excessive padding [1]. A stride of 1 is used [1]. Sequences shorter than the window length are padded to a uniform length of 20 using a dedicated padding token; padding tokens are masked during training to prevent the model from treating padding as an anomalous signal [18],[27],[28]. For frequency-based models, event occurrence vectors are max normalized [20].

Class Imbalance Handling:

For the supervised Random Forest model, class imbalance is addressed through two data-level resampling strategies, each evaluated as a separate experimental condition:

1. Balanced undersampling- normal training blocks are randomly reduced to match the number of anomalous training blocks, giving training set of equally 6000 normal and anomalous blocks, following established practice [18] [28] [3].

2. Random oversampling- anomalous training blocks are randomly duplicated with replacement to match count with the normal class, producing a training set of approximately 390,770 samples per class. increasing minority class representation without discarding majority class samples, following the methodology of Alsalmi et al. [9].

Synthetic oversampling with SMOTE is excluded. With log data reducing 575,061 sequences to only 26,814 unique patterns, simple duplication achieves the same performance gains without introducing unnatural event sequences [9]. Moreover, interpolation by SMOTE can introduce artificial sequences that do not correspond to real execution paths [11], [22], while random duplication is both computationally lighter and empirically effective. Studies have proved other techniques to be better without the expensive computational cost of SMOTE [9].

For the semi-supervised deep learning models, class imbalance is not addressed at the training level since these models are designed to be trained exclusively on normal log sequences. Anomalous sequences not being in training data keeps models unaffected by the skewed ratio. Including anomalous blocks during training would cause reconstruction-based models such as the Autoencoder and LSTM Autoencoder to reconstruct anomalous blocks. For the LSTM, adding anomalous event sequences to next-event prediction training would contaminate the learned normal transition grammar, degrading the top-g anomaly detection rule. Two conditions are compared again to tackle class imbalance for reconstruction models: (i) Normal only training, mixed validation and test sets [1] [27] [6] (ii) Normal only training, balanced classes validation set, mixed test set [24]

Both conditions have threshold calibrated to obtain best results.

The LSTM model is excluded from this comparison as its top-g detection rule is fixed and independent of validation set composition, requiring no threshold selection from the validation set. As a consequence of the imbalance, accuracy alone is an insufficient evaluation metric. The primary metrics for this study are F1-score, Precision, Recall, False Positive Rate, and PR-AUC, which remain informative under class imbalance [21], [13].

The LSTM model is excluded from this comparison as its top-g detection rule is fixed and independent of validation set composition, requiring no threshold selection from the validation set.

As a consequence of the imbalance, accuracy alone is an insufficient evaluation met-

ric. The primary metrics for this study are F1-score, Precision, Recall, False Positive Rate, and PR-AUC, which remain informative under class imbalance [13], [21].

Model	Condition 1	Condition 2
Random Forest	Balanced undersampling ($\sim 6k$ / $6k$)	Random oversampling ($\sim 390k$ / $390k$)
AE	Normal-only training + natural validation threshold	Normal-only training + balanced validation threshold
LSTM-AE	Normal-only training + natural validation threshold	Normal-only training + balanced validation threshold
LSTM	Normal-only training, $g = 9$	No second condition. Balanced val has no effect since detection rule is fixed at $g=9$

Table 4.6: Two Experimental Conditions per Model to Handle Class Imbalance

Table 4. shows the experimental conditions for each model. Each model is evaluated under two conditions to test the effect of class imbalance handling, except the LSTM, whose fixed detection rule makes a second condition unnecessary

4.2.3 Model Design Paradigms and Selection Rationale

As discussed in Chapter 2, log anomaly detection encompasses a wide range of approaches including frequency-based, sequence-based, semantic-based, graph-based, reconstruction-based, context-based, LLM-based, parsing-free, and federated learning methods. For this study, model is selected by compatibility with HDFS_v1 logs and the limitations of evaluation scope. Graphs-based, federated solutions, LLM-based approaches are excluded due to their computational expense and implementation complexity [9], [11], [15].

This study evaluates four models to cover three distinct anomaly detection paradigms: sequence-based, reconstruction-based, and frequency-based. This selection allows direct comparison between deep learning and classical machine learning approaches under identical preprocessing and evaluation conditions, following the works of Yu et al. [4] and Ali et al. [6].

Model 1 : LSTM (Sequence-based)

The LSTM model is included as the canonical sequence-based baseline, directly following the DeepLog architecture of Du et al. [1], which established next-event prediction as the main paradigm for log anomaly detection. The model learns the normal sequence events of HDFS execution traces by training on the sequences of event IDs. A block is flagged as anomalous, if any predicted next event falls outside the top- g most probable candidates under the learned distribution. LSTMs are well suited for HDFS_v1 because its block-level traces represent natural ordered execution paths, making sequential modelling directly applicable [1], [19]. A limitation of

LSTM-based next-event prediction is that it may miss anomalies where the event sequence order appears normal but event frequencies are unusual, since it does not model quantitative patterns [27].

Model 2 : Autoencoder (Reconstruction-based , Quantitative)

The Autoencoder is included as the reconstruction-based model for frequency-based anomaly detection, following Farzad and Gulliver [30] and the survey of Landauer et al. [13]. The model is trained on normal block-level event occurrence vectors and it learns to reconstruct those frequency distributions. When reconstruction error exceeds threshold, it is an anomaly. A key operational advantage in the SIEM context is that the Autoencoder can be trained on unlabelled normal data, making it usable in environments where anomaly labels are unavailable [13], [30]. The boundary between normal reconstruction error and anomalous reconstruction error must be determined empirically on validation data [13], [22].

Model 3 : LSTM Autoencoder (Sequence-based, Reconstruction-based)

The LSTM Autoencoder combines the sequential memory of LSTM layers with the reconstruction-based anomaly scoring of the Autoencoder, following Bekkouche et al. [24] and Aziz and Munir [10]. The encoder part of the model compresses normal log sequences into a latent representation and the decoder reconstructs back to the original sequence. Because both encoder and decoder are LSTM-based, the model captures both event order and event co-occurrence patterns simultaneously, giving it a theoretical advantage over the plain Autoencoder on sequence anomalies and over the plain LSTM on frequency anomalies. Like the Autoencoder, it can be trained on unlabelled normal data and is applicable in SIEM contexts where labels are scarce [24], [10]. Limitations include greater computational cost than the plain Autoencoder, and the requirement for careful padding and masking of variable-length sequences to avoid treating padding tokens as anomalous events [13], [21], [22].

Model 4 : Random Forest (Classical Supervised Classification)

Random Forest is included as a classical supervised model, following the empirical findings of Yu et al. [4] and Ali et al. [6], who state that classical machine learning methods remain competitive with deep learning approaches for log-based anomaly detection while requiring significantly less training time and being less sensitive to hyperparameter tuning. By including RF, this study allows to directly test whether the added complexity of deep learning models produces operational benefits in a SIEM-style evaluation, or whether a simpler classical classifier achieves comparable detection quality with lower computational overhead. RF uses the same 29-dimensional max-normalized event frequency vectors as the Autoencoder, treat-

ing each block as a fixed-length feature vector. Unlike the three deep learning models which train on normal data only, RF is a supervised classifier that requires examples from normal and anomalous classes during training. Class imbalance is handled during training through two resampling conditions which are balanced undersampling and random oversampling rather than at the thresholding stage, as described in Section 4.2.2.

4.3 Model Architecture Specification

4.3.1 Feature representations

LSTM

Input files: Event_traces.csv, HDFS_templates.csv Feature representation: Sequential Event_ID sequence. Each sequence within a window, if smaller than window length, is padded.

Block traces represent the execution path as an ordered list of Event_IDs. LSTM trains by predicting the next event in the sequence. Lengths of each trace sequence varies according to window size selected hence the need for padding (lengthening short traces), to feed into neural networks, since those require the same length inputs. Since in HDFS short sequences are often anomalous, padding has to be done carefully and only when necessary. Padding can introduce artificial patterns since unique tokens for the padding may lead to models identifying the padding as anomalous, which is not desirable [13], [21], [22].

Autoencoder

Input files: Event_occurrence_matrix.csv, HDFS.npz

Each block is represented as a vector of fixed length, that record count of occurrences of each event type. Order is ignored since anomalies within HDFS often seem to be unusual event counts rather than unusual event orders. Vectors are normalized to not allow frequent events to dominate reconstruction. This ensures rare and important anomalies are not missed. No padding is done since all vectors are of fixed length.

LSTM Autoencoder (Sequence-based)

Input files: Event_traces.csv, HDFS_templates.csv Representation: Ordered Event_ID sequences, padded for uniform length.

Encoder LSTM compresses the sequence into a hidden representation; decoder LSTM reconstructs the original sequence. Reconstruction error is used as an anomaly score. Padding and masking are required for variable-length sequences.

4.3.2 Dataset split

All deep learning models have training sets that contain only normal traces of label 0, and the validation/Test set contains both normal and anomalies (label 0, label 1). For all models, splitting is strictly done block-wise to avoid data-leakage, so the models do not unfairly learn traces that appeared in both train and test sets. Chronological split is avoided since there are no clear time stamps for the parsed, preprocessed HDFS_v1. For the LSTM sliding window is used of length 20, stride is of 1 is applied within each block trace to predict next event. Stride of 1 is since it has been standard practise in log anomaly detection studies, to ensure model's maximum learning of event transitions and variety of sequential patterns within each trace [1]. The LSTM Autoencoder and Autoencoder do not use sliding windows since each block is treated as a vector sequence respectively. Data is split 70/15/15 for train, validation and test sets.

LSTM: The model is trained on normal traces so it is able to learn the normal event-sequence structure and detect deviations. Validation and test sets include anomalies to evaluate detection capability.

Autoencoder: The model is trained only on normal block-level event-occurrence vectors and learns the normal frequency patterns of system behaviour. Upon testing, when reconstruction error is high enough above the threshold, traces are flagged as anomalous. If the same block's frequency vector appeared in both the train and test sets, the autoencoder will trivially reconstruct it.

LSTM Autoencoder: The model is trained on normal log sequences. The encoder part compresses and the decoder part reconstructs event order. Anomalies are detected when reconstruction error exceeds the set error threshold.

Random Forest learns from all training blocks including both normal and anomalous, since it is a supervised classifier requiring labelled examples from both classes. The same 70/15/15 block-level split is used.

4.3.3 LSTM

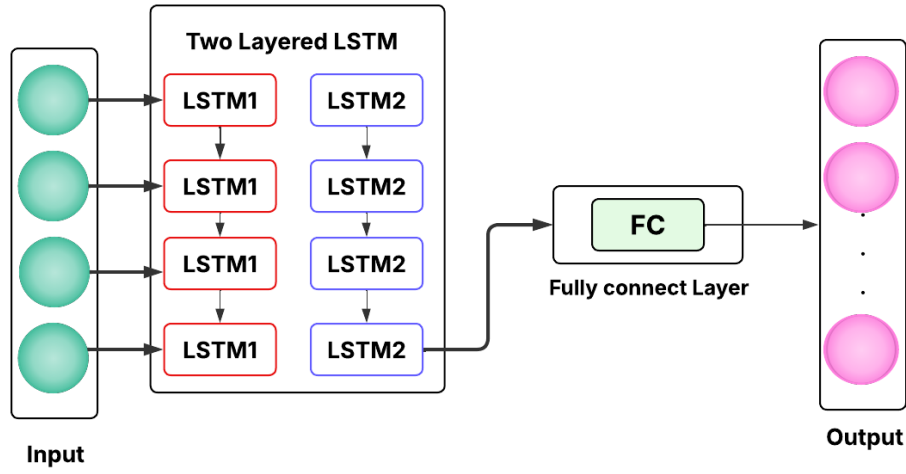


Figure 4.10: Double-stack LSTM Architecture Used for next-event Prediction adapted from [1]

Input format:

- Sequential Event_ID sequences are constructed from Event_traces.csv and HDFS.log_templates.csv files.
- Sequences are grouped block-wise, then sliding windows of fixed length 20 events are applied within each block trace for next-event prediction.
- Shorter window sequences are padded with padding tokens that are masked during computation, so the padding is not treated as a meaningful event [13], [21], [22].

Core layers:

- Embedding layer to convert 29 Event.IDs into dense vectors with dimension 32.
- Two stacked LSTM layers (DeepLog design [1]), with 64 hidden units each, capturing sequential dependencies and long-range event order.
- Fully connected output layer with softmax activation that produces a probability distribution for each known event template.

Output:

- The model predicts the next event in each window. An anomaly is flagged when the predicted event does not match any of the top-g events (top options of the events with the highest probabilities of following the current event)

Block-level aggregation:

- To avoid inflating alert counts due to multiple windows within the same block, a block is flagged anomalous if any window within it is marked as anomaly under the top- g rule, such that each block contributes to at most one alert in the final evaluation, consistent with Du et al. [1].

Loss function:

- Categorical cross-entropy loss between the predicted event distribution and the actual next event.

Key Hyperparameters	Value
Sequence Length	20 (since HDFS traces have an average of 19 per block)
Stride	1
Embedding dimension	32
LSTM Hidden Units	64 units per layer
Early-stop	Patience of 3
Learning Rate	0.001
Optimizer	Adam
Candidate set size (g)	9 (predicts top- g most likely next events)
Loss Function	Categorical cross-entropy

Table 4.7: Hyperparameters of the LSTM model

4.3.4 Autoencoder

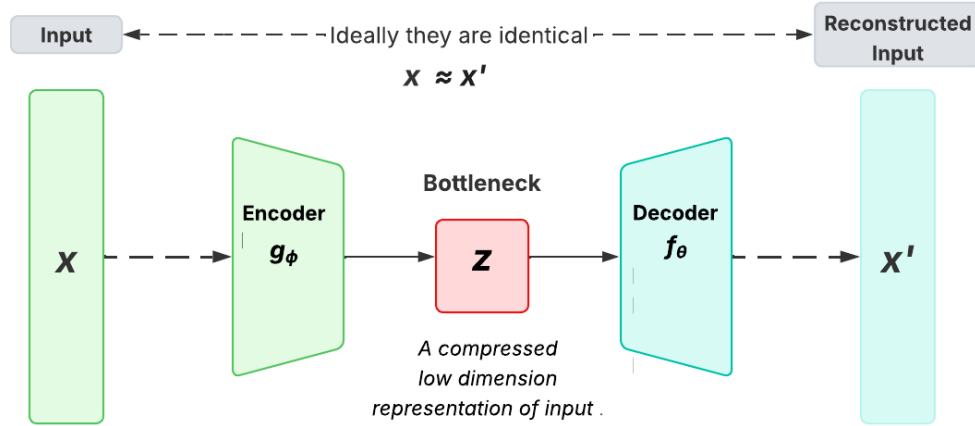


Figure 4.11: Autoencoder architecture, adapted from: Weights & Biases, "An Introduction to Adversarial Latent Autoencoders," 2021. [Online]. Available: <https://wandb.ai/authors/alaie/reports/An-Introduction-to-Adversarial-Latent-Autoencoders--VmlldzoxNDA2MDY>

Input format:

- Event occurrence vectors from Event_occurrence_matrix.csv and HDFS.npz. Each block represented as a fixed-length 29 dimensional vector of event counts. No padding is required since all vectors are of identical length.

Core layers:

- Encoder: Dense layers progressively reduce dimensionality of input vectors.
- Bottleneck: Compressed latent representation of normal frequency patterns.
- Decoder: Dense layers reconstruct the original event occurrence vector.

Output / anomaly score:

- Reconstruction error is the mean squared error between input and output vectors. An anomaly is flagged when the reconstruction error exceeds the anomaly detection threshold. An anomaly is flagged when the reconstruction error exceeds the anomaly detection threshold, which is set at the F1-optimized threshold of reconstruction errors computed on normal validation blocks only. Order of events is ignored; only event counts are reconstructed.

Loss function:

- Mean squared error (MSE).

Key Hyperparameters	Value
Latent Dimension	16
Encoder/decoder depth	2 layers each
Hidden Layer activation	ReLU
Output Layer activation	Linear
Learning Rate	0.001
Optimizer	Adam
Loss Function	Mean Squared Error(MSE)
Threshold Selection	F1-optimized of reconstruction errors on normal validation blocks

Table 4.8: Hyperparameters of the Autoencoder Model

4.3.5 LSTM Autoencoder

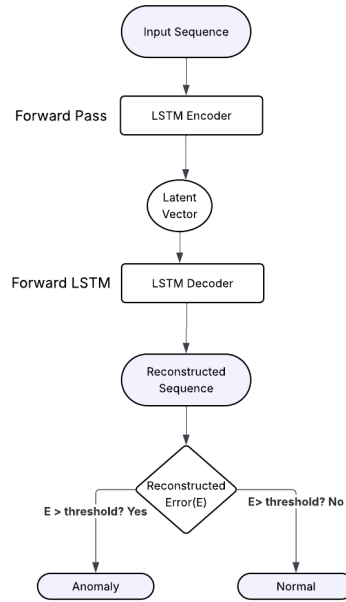


Figure 4.12: LSTM AE architecture, adapted from Bekkouche et. al [24]

Input format:

- Ordered Event_ID sequences from Event_traces.csv and HDFS_templates.csv. Sequences truncated/padded and padding tokens masked.

Core layers:

- Encoder: Stacked LSTM layers compress input sequence into latent vector.
- Bottleneck: Latent representation captures sequential dependencies.

- Decoder: LSTM layers reconstruct the original sequence from latent vector.

Output / anomaly score:

- Reconstruction error is computed as the mean sparse categorical cross-entropy between the original integer-encoded event sequence and the reconstructed sequence. An anomaly is flagged when the error exceeds the threshold.

Key Hyperparameter	Value
Sequence Length	20
LSTM Hidden Units	64
Latent dimension	64
early-stop	Patience of 3
Learning rate	0.001
Optimizer	Adam
Loss function	Sparse categorical cross-entropy
Threshold selection	Two thresholds reported: F1-optimized and SIEM-constrained, both selected on normal validation blocks only

Table 4.9: Hyperparameters of the LSTM-AE Model

4.3.6 Random Forest

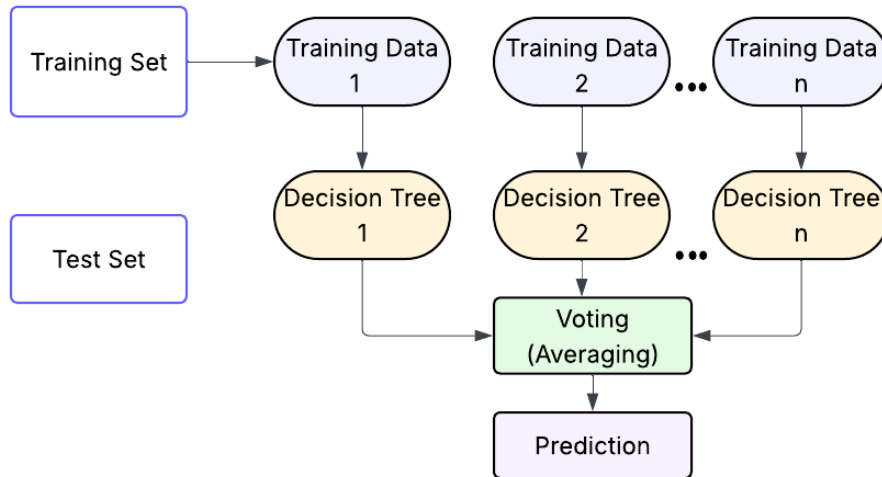


Figure 4.13: Random Forest Architecture

Input format:

- Each HDFS block is represented as a fixed-length 29-dimensional max-normalized vector of event counts from Event_occurrence_matrix.csv and HDFS.npz. Unlike sequence-based models, no sequential ordering is used. Each block is treated as an independent fixed-length feature vector.

Output / anomaly score:

- The Random Forest model outputs the predicted probability of the anomalous class using the predict_proba() function. The class-1 probability is used as a continuous anomaly score for ROC and Precision–Recall curve generation.

Key Hyperparameters	Value
Number of trees (n_estimators)	200
Maximum tree depth (max_depth)	None
Minimum samples per leaf (min_samples_leaf)	1
Class weighting	Two conditions: balanced undersampling ($\sim 6k/6k$) and random oversampling ($\sim 390k/390k$)
Threshold selection	F-1 optimized on validation set
Random seed (random_seed)	42

Table 4.10: Hyperparameters of the Random Forest Model

4.3.7 Evaluation Plan (SIEM-Style)

This study evaluates log anomaly detection models using a SIEM-style perspective, where model outputs are interpreted as alerts that would be consumed by SOC analysts. In SOC, analysts often receive more alerts than they can investigate, especially when false positives are high. Each positive prediction i.e. an anomaly can be treated as an analyst-facing alert. This framing is motivated by SOC operational reality where excessive false alerts overwhelm analysts causing alert fatigue. Threats which need immediate attention are overlooked due to overload [20]. Therefore, in a SIEM-style framing, false positives are not just an accuracy issue, they directly translate into wasted analyst time and operational risk [20].

Evaluation is conducted on the HDFS dataset at the block level, consistent with the dataset’s block-wise grouping and labels introduced in prior work and provided through the standardized LogHub preprocessing [1], [26]. For each block, the model

outputs a binary decision (normal/anomalous) after applying its anomaly scoring rule and decision threshold. We compute a confusion matrix with TP , FP , FN , and TN and report the following metrics:

Precision: Precision measures among raised alerts, how many are true anomalies.

$$\text{Precision} = \frac{TP}{TP + FP} \quad (4.1)$$

Recall (Sensitivity/ TPR): Recall measures among true anomalies, how many are successfully detected .

$$\text{Recall} = \frac{TP}{TP + FN} \quad (4.2)$$

F1-score: The F1-score is the single summary metric that is high only when both precision and recall are high; useful under class imbalance.

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (4.3)$$

False Positive Rate (FPR): FPR is treated as a primary SIEM-oriented metric because false positives directly increase false alerts and contribute to alert fatigue in SOC operations [14].

$$\text{FPR} = \frac{FP}{FP + TN} \quad (4.4)$$

Specificity (True Negative Rate, TNR) : TNR measures how well the model ignores raising alerts on normal blocks. If the value of TNR is high, it means fewer false alerts were generated, which is preferred for SOC workloads.

$$\text{TNR} = \frac{TN}{TN + FP} \quad (4.5)$$

Alert Volume : It is the total number of predicted anomalous blocks.

$$\text{Alert Volume} = TP + FP \quad (4.6)$$

ROC curve: ROC curve is plotted for models that output a continuous anomaly score like a scalar anomaly score for sequence models or reconstruction error for Autoencoder/ LSTM-AE. We can evaluate performance across all possible thresholds by plotting the ROC curve. The ROC curve plots the True Positive Rate (TPR) against the False Positive Rate (FPR):

$$\text{TPR} = \frac{TP}{TP + FN}, \quad \text{FPR} = \frac{FP}{FP + TN} \quad (4.7)$$

This is useful because SOC/SIEM deployments often operate under different alert-

budget constraints, so a single fixed threshold may not capture the full trade-off.

AUC-ROC : We report the area under the ROC curve (AUC), to summarize the ROC curve into a single threshold-independent number.

$$\text{AUC-ROC} = \int_0^1 \text{TPR}(\text{FPR}) d(\text{FPR}) \quad (4.8)$$

Model-specific alert generation: For the LSTM (DeepLog-style), next-event prediction is performed using sliding windows and a top- g rule for unexpected events [1]. To avoid inflating alert counts due to multiple windows within the same block, window-level anomaly evidence is aggregated into a single block-level decision so each block contributes at most one alert in the main evaluation.

For the Autoencoder and LSTM Autoencoder, anomaly detection is based on reconstruction error: after training on normal data, each block is reconstructed and the reconstruction error is used as the anomaly score; higher error indicates higher likelihood of anomaly [18]. A threshold is required to convert continuous scores into binary decisions (alert vs no alert). Threshold is selected using the validation set and fixed before evaluation on the test set to avoid tuning decisions on test data.

Two threshold settings are reported:

1. an F1-optimized threshold that maximizes validation F1; and
2. a SIEM-oriented threshold chosen to satisfy an operational constraint on the validation set to control false alerts, motivated by alert-fatigue concerns [14].

Although we do not integrate with a SIEM platform, we discuss practical considerations relevant to deployment. Specifically, we report:

- training time (per epoch / total) and
- inference throughput (blocks per second)

as indicators of scalability on the full HDFS dataset [13]. These considerations matter because SOC workflows are not only about accuracy, but also if the alerts generated are manageable. Together, these choices evaluate both detection capability and operational usability of the models as alert generators in SOC-style monitoring workflows [14].

In real SOC environments, analysts often get overwhelmed by large volumes of alerts which are generated by SIEM systems. Not all alerts require the same level of attention. From an operational perspective, the anomaly scores produced by deep

learning models can conceptually support alert prioritization. Based on the magnitude of the anomaly score or reconstruction error alerts can be mapped to different risk or severity levels like low, medium and high. Higher scores may indicate more severe or urgent anomalies which need immediate attention to prevent further security threat, while lower scores may correspond to low-risk alerts.

For Random Forest, the model is trained on labelled blocks represented as 29-dimensional max-normalized event frequency vectors, and the predicted probability of the anomalous class is used as a continuous anomaly score [4], [6]. Each block that is predicted as an anomaly is treated as analyst-facing alert, consistent with the block-level alert framing that is applied to all models.

4.4 Implementation of Selected design

All models are implemented in Python. TensorFlow/Keras used for deep learning models and scikit-learn for Random Forest. Experiments were conducted on Google Colab with approximately 12GB RAM and T4 GPU support. All models share a common data pipeline loading from the same preprocessed HDFS_v1 files to training and testing the models. A block-level split of 70/15/15 was created using a fixed random seed of 42 and saved as a shared JSON file loaded by all four model notebooks. Training, validation, and test sets were constructed from this shared split for each model according to its input representation; sequential event ID sequences for LSTM and LSTM-AE, normalized frequency vectors for Autoencoder and Random Forest.

A shared utility module `siem_utils.py` was implemented to compute all metrics identically across all four models. That includes accuracy, precision, recall, F1-score, FPR, TNR, alert volume, AUC-ROC, PR-AUC, training time, inference time and throughput. Full experimental results are presented in Chapter 5.

Chapter 5

Result Analysis

This chapter contains the experimental results obtained from evaluating the models - Autoencoder, Random Forest, LSTM and LSTM-AE on the HDFS_v1 benchmark dataset under the SIEM-style evaluation framework defined in Chapter 4 with two class imbalance handling conditions. The chapter reports and compares detection performance for each model individually, reports threshold decisions, shows statistical comparisons, and discusses the implications for SIEM deployment along with key limitations.

5.1 Experimental Setup

All experiments were conducted on the HDFS_v1 benchmark dataset from the LogHub repository [20], using the block-level 70/15/15 train/validation/test split described in Section 4.2.2. For models requiring threshold-based binary classification (Autoencoder and LSTM-AE), candidate thresholds were selected exclusively on the validation set prior to any test evaluation, to prevent threshold overfitting [6]. All evaluation metrics, including precision, recall, F1-score, false positive rate (FPR), true negative rate (TNR), alert volume, AUC-ROC, PR-AUC, training time, and inference throughput, were computed using a single shared evaluation function in `siem_utils.py`.

5.2 Performance Evaluation

This section contains the experimental results for each model. As defined in Section 4.2.2, all the models are evaluated under two conditions except LSTM. The two conditions show different ways of how class imbalance is handled. For Random Forest, balanced undersampling (Condition 1) versus random oversampling (Condition 2) ; and for the Autoencoder and LSTM-AE, a normal validation threshold (Condition

1) versus a balanced validation threshold (Condition 2). LSTM is evaluated under a one condition, as balanced validation has no effect because the detection rule is fixed at $g=9$. Table 5.1 and 5.3 reports the results for Condition 1, and Table 5.2 and 5.4 reports the results for Condition 2.

Model	Accuracy	Precision	Recall	F1-Score	AUC-ROC	PR-AUC
Autoencoder	0.9842	1.0000	0.4552	0.6256	0.9994	0.9859
LSTM	0.9545	1.0000	0.0533	0.1011	0.5266	0.0988
LSTM-AE(F1-thres.)	0.9810	0.6473	0.7593	0.6988	0.9724	0.6492
LSTM-AE(SIEM-thres.)	0.9528	0.3703	0.8900	0.5230	0.9724	0.6492
Random Forest	0.9991	0.9784	0.9912	0.9848	0.9986	0.9824

Table 5.1: Traditional Detection Metrics of All Models on HDFS_v1 Test Set with Condition 1 from Table 4.6

Table 5.1 shows on the Condition 1 test set, the Random Forest has the strongest overall detection performance, with an accuracy of 0.9991, precision of 0.9784, recall of 0.9912, and the highest F1-score (0.9848) and PR-AUC (0.9824) among all models. The Autoencoder records a higher AUC-ROC (0.9994) than the RF (0.9986), its recall of just 0.4552 means it misses more than half of all anomalies, giving it the lowest F1-score (0.6256) of any model. This contrast shows why AUC-ROC and accuracy are insufficient in this setting. The LSTM-AE sits in the middle (F1 0.6988 at the F1-optimized threshold, 0.5230 at the SIEM threshold), while the LSTM is the weakest detector, with a recall of only 0.0533 and a near-random PR-AUC of 0.0988.

Model	FPR	Specificity (TNR)	Alert Vol.	Training Time (s)	Inference Time (s)	Throughput (blk/sec)
Autoencoder	0.0000	1.0000	1,142	118.10	5.14	16,780
LSTM	0.0000	1.0000	54	173.48	2.1	10,052
LSTM-AE(F1- thres.)	0.0124	0.9876	2,943	183.26	3.42	24,712
LSTM- AE(SIEM- thres.)	0.0453	0.9547	6,030	183.26	3.42	24,712
Random For- est	0.0007	0.993	2,542	1.05	1.37	62,735.0

Table 5.2: SIEM-Oriented Evaluation Metrics of All Models on HDFS_v1 Test Set with Condition 1 from Table 4.6

Table 5.2 shows under the SIEM-oriented evaluation for Condition 1, the Random Forest records the lowest FPR of 0.0007 and a high TNR of 0.993, alert volume of 2,542 against the 2,509 true anomalies, training in 28.15s, the fastest inference (1.89 s), and the highest throughput at 45,612 blocks per second. The Autoencoder achieves a perfect TNR of 1.0000 and zero FPR but raises only 1,142 alerts and processes 16,780 blocks per second. The LSTM likewise produces no false alarms (FPR 0.0000, TNR 1.0000), but its 54 alerts reflect its near-zero recall rather than good performance, and it is the slowest at inference (10,052 blocks per second). The LSTM-AE shows the clearest threshold trade-off: the F1-optimized threshold keeps FPR low (0.0124, TNR 0.9876) with 2,943 alerts, while the SIEM-constrained threshold raises FPR to 0.0453 (TNR 0.9547) and inflates alert volume to 6,030 which more than double the true anomaly count, at 24,712 blocks/sec.

Model	Accuracy	Precision	Recall	F1- Score	AUC- ROC	PR- AUC
Autoencoder	0.9998	1.0000	0.456	0.6263	0.9998	0.9918
LSTM	0.9545	1.0000	0.0533	0.1011	0.5266	0.0988
LSTM-AE(F1- thres.)	0.9816	0.7063	0.6277	0.6647	0.9733	0.6213
LSTM- AE(SIEM- thres.)	0.9586	0.4037	0.8876	0.555	0.9733	0.6213
Random For- est	0.9999	0.9976	0.9992	0.9984	1.0000	1.000

Table 5.3: Traditional Detection Metrics of All Models on HDFS_v1 Test Set with Condition 2 from Table 4.6

Table 5.3 shows for condition 2, the perfect AUC-ROC and PR-AUC values with accuracy of 0.9999, precision of 0.9976, recall of 0.9992 and the highest F1 score of 0.9984 are achieved by Random Forest. The LSTM-AE (AUC-ROC 0.9733, PR-AUC 0.6213 for both thresholds) shows the recall–precision trade-off: the F1-optimized threshold gives F1 0.6647 (precision 0.7063, recall 0.6277), while the SIEM threshold raises recall to 0.8876 but drops precision to 0.4037, lowering F1 to 0.555. The Autoencoder achieves F1 0.6263, recall 0.456 and perfect precision, but the AUC-ROC is quite high (0.9998), indicating that the model has overstated its performance in the presence of imbalance. As before the LSTM is the weakest (recall = 0.0533, PR-AUC = 0.0988), proving that it is not suitable for this dataset.

Model	FPR	Specificity (TNR)	Alert Vol.	Training Time (s)	Inference Time (s)	Throughput (blk/sec)
Autoencoder	0.0000	1.0000	1,144	59.47	7.25	11,891.0
LSTM	0.0000	1.0000	54	173.48	2.1	10,052
LSTM-AE(F1- thres.)	0.0078	0.9922	2,230	127.03	2.38	36,178.0
LSTM- AE(SIEM- thres.)	0.0393	0.9607	5,516	127.03	2.38	36,178.0
Random For- est	0.0001	0.999	2,513	80.75	0.54	160,724.0

Table 5.4: SIEM-Oriented Evaluation Metrics of All Models on HDFS_v1 Test Set with Condition 2 from Table 4.6

Table 5.4 shows SIEM-oriented evaluation for Condition 2, the Random Forest has FPR of 0.0001 and the TNR of 0.999, alert volume of 2,513 against the 2,509 true anomalies. It also has the highest throughput 160,724 blocks processed per second, with the fastest training (80.75 s) and inference (0.54 s) times. The Autoencoder achieves a perfect TNR of 1.0000 and zero FPR, raising only 1,144 alerts, but processes just 11,891 blocks per second. The LSTM likewise produces no false alarms (FPR 0.0000, TNR 1.0000) but raises only 54 alerts, a direct consequence from its low recall and throughput (10,052 blocks per second). The LSTM-AE shows the clearest threshold trade-off: at the F1-optimized threshold it keeps FPR low (0.0078, TNR 0.9922) with 2,230 alerts, whereas the SIEM-constrained threshold pushes FPR to 0.0393 and TNR down to 0.9607, inflating alert volume to 5,516 that is more than double the true anomaly count, at a throughput of 36,178 blocks per second.

5.2.1 Autoencoder

Normal-only training and Normal validation threshold

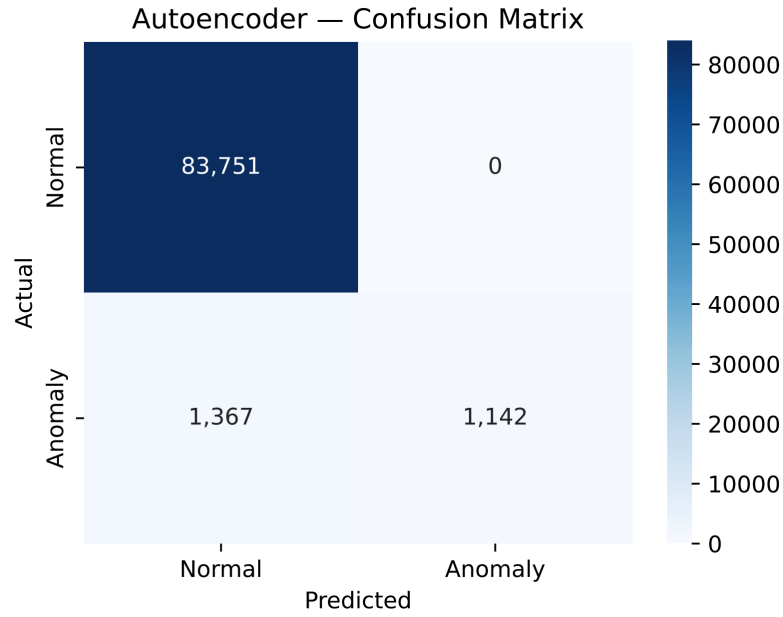


Figure 5.1: Confusion Matrix for the Autoencoder Model

Figure 5.1 shows confusion matrix shows of autoencoders. The model was correctly able to identify 83,751 normal events as normal, 0 normal were identified as anomaly, 1,367 anomalies were predicted as normal and 1,142 actual anomalies were identified.

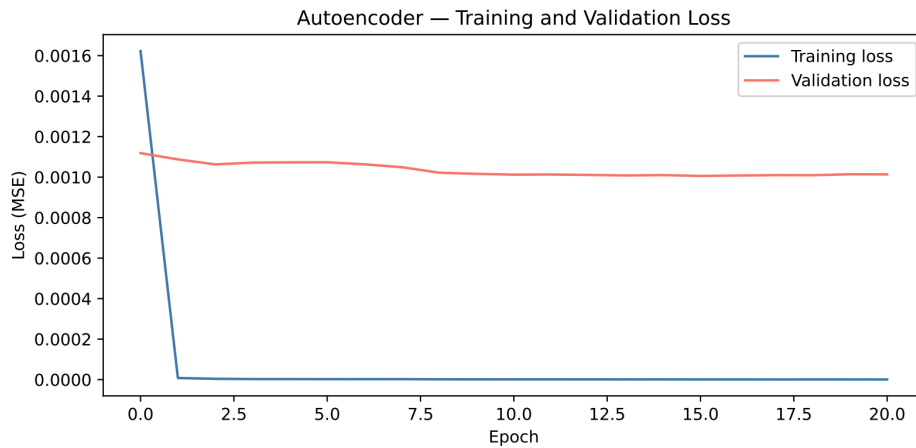


Figure 5.2: Training and Validation Loss for the Autoencoder Model

Figure 5.2 shows that training loss dropped to zero after two training sessions. Training loss did not change significantly after the two training sessions. The autoencoder captures normal behaviour. However, memorising leads to overfitting the model. The autoencoder did not identify blocks that were slightly outside the normal training threshold.

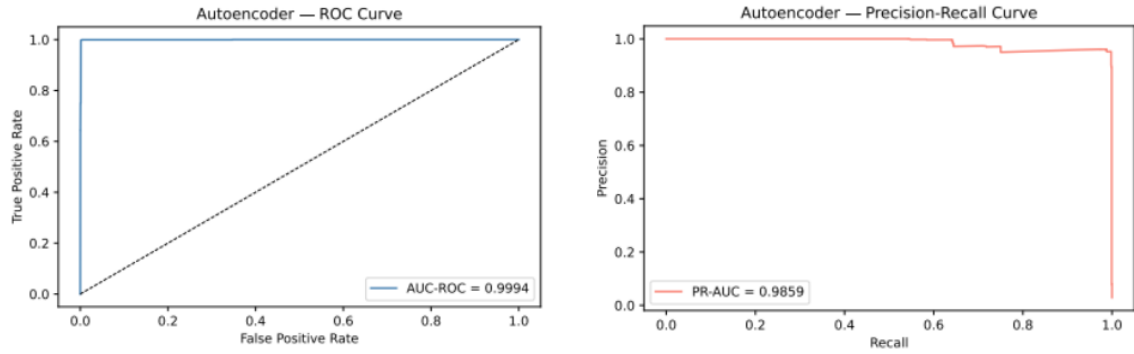


Figure 5.3: ROC and Precision-Recall curves for the Autoencoder Model

The autoencoder ROC-AUC and precision-recall curves are presented in Figure 5.3. The near-perfect ROC curve (AUC-ROC 0.9994) and precision-recall curve (PR-AUC 0.9859) show that the autoencoder captured reconstruction error scores and ranked these scores along the thresholds. The ROC curve shows that the autoencoder is capable of identifying anomalies, and the detection failure lies on the threshold.

Normal-only training and balanced validation,SIEM Threshold

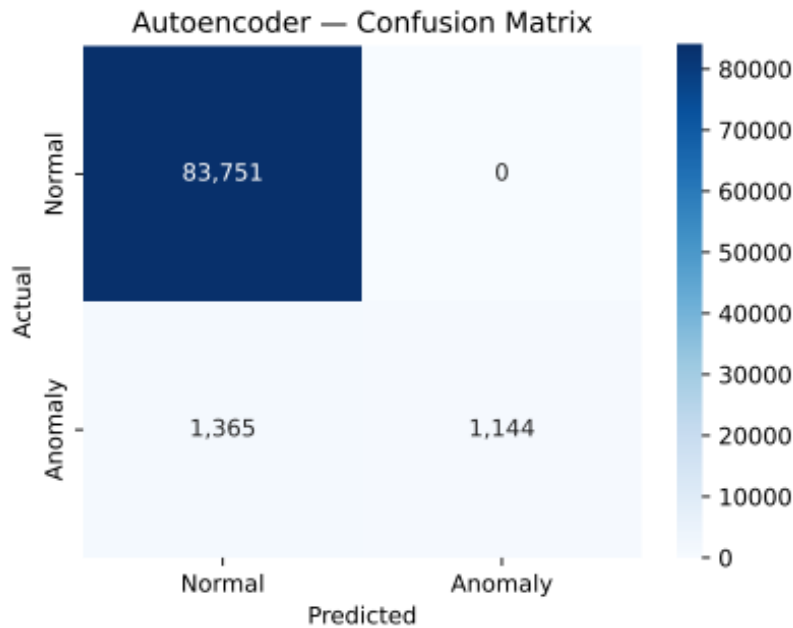


Figure 5.4: Confusion Matrix for the Autoencoder Model

Under the balanced-validation condition, the Autoencoder correctly classified all 83,751 normal blocks and raised zero false positives, while detecting 1,144 of 2,509 anomalies and missing 1,365. This gives a precision of 1.0000, recall of 0.456, and an F1-score of 0.6263, with an alert volume of 1,144, every alert being a genuine anomaly. The perfect precision means no analyst time is wasted on false alarms, but

the recall of 45.6% shows that more than half of all true anomalies go undetected, leaving 1,365 faults silently missed.

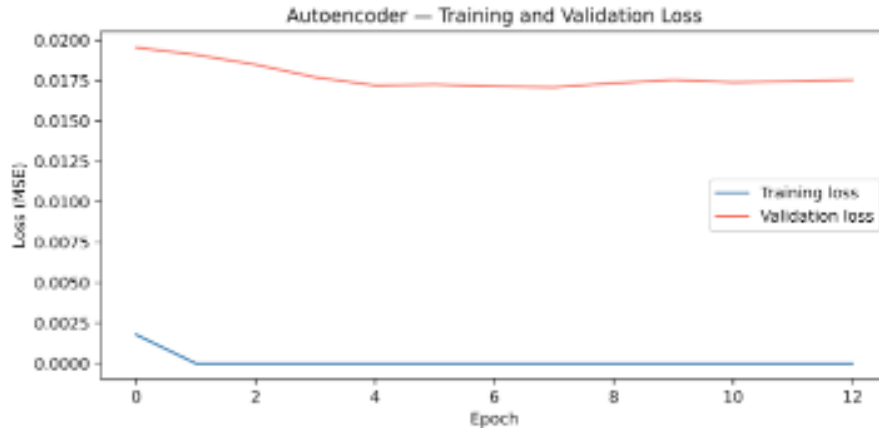


Figure 5.5: Training and Validation Loss Curves for AE

Fig.5.5 shows that the training loss becomes almost zero within two epochs and stays flat, while the validation loss settles around 0.017–0.018 and remains stable. This shows the Autoencoder learns normal patterns quickly and does not overfit, but the steady gap reflects its limited ability to flag anomalies that deviate only slightly from normal behaviour.

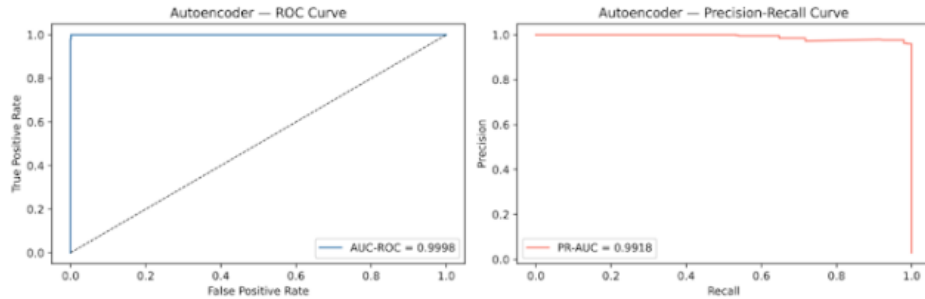


Figure 5.6: ROC-AUC and PR curve for AE

The figure tells us the model can separate anomalies well. The ROC curve is almost a straight line towards the top left corner of the plot, and the AUC-ROC value is 0.9998, indicating that the Autoencoder almost always assigns a higher reconstruction error to blocks that are considered anomalous blocks than to normal blocks. The Precision-Recall curve remains flat over most of the recall range with a PR-AUC of 0.9918, which is a more meaningful measure in this instance due to the highly imbalanced nature of the data set (only 2.93% anomalies). Lower threshold would mean increased number of anomalies (but more false alarms).

5.2.2 Random Forest

Balanced undersampling

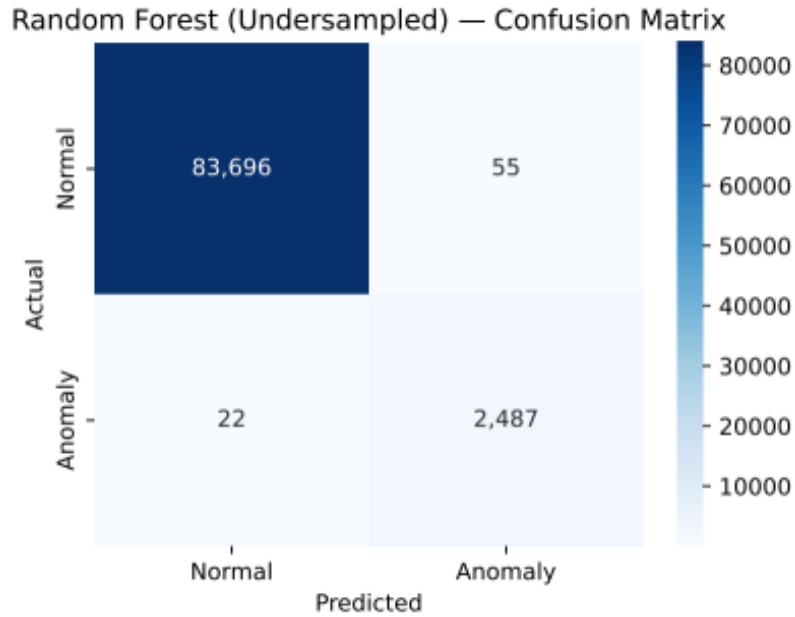


Figure 5.7: Confusion Matrix for Random Forest with Undersampling

Fig 5.7 shows the confusion matrix of RF with balanced undersampling. The model was able to detect 83,696 of 83,751 normal blocks, and identified 2,487 of 2,509 anomalies, with 55 false positives and 22 false negatives. This led to a precision of ~ 0.978 , a recall of ~ 0.991 and a slightly lower F1 score than oversampling condition but has alert volume of 2,542. The increase in false positives (from 6 to 55) reflects the cost of removing most normal training data: with fewer normal examples to learn from, the decision boundary becomes slightly less precise, raising the analyst false-alarm burden.

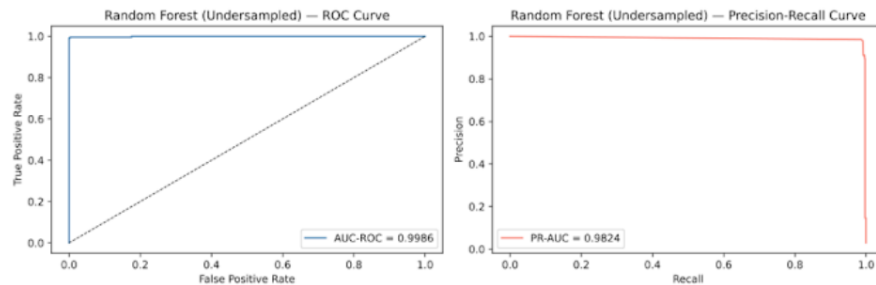


Figure 5.8: ROC and Precision-Recall Curves for the RF Model

Fig 5.8 shows the curves an AUC-ROC of 0.9986 and a PR-AUC of 0.9824. The precision-recall curve is seen to have a slight decline at high recalls, which indicates that the model may generate more false positives at high recall rates. Overall, the

performance remains good, but with the use of undersampling, using around 6,000 normal data, the precision becomes lower.

Random Oversampling

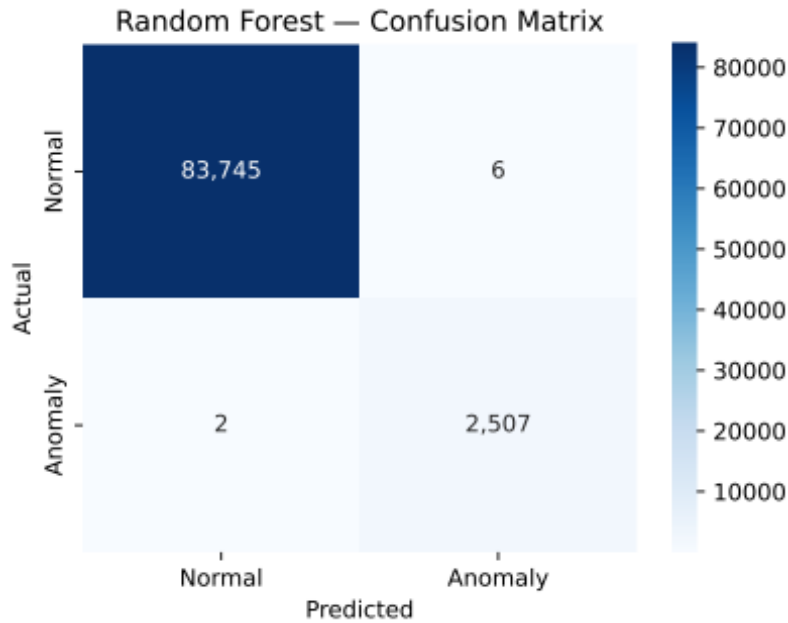


Figure 5.9: Random Forest ROC and Precision-Recall Curves (Oversampling)

Fig 5.9 shows the confusion matrix with under random oversampling, the Random Forest correctly classified 83,745 of 83,751 normal blocks and detected 2,507 of 2,509 anomalies, producing only 6 false positives and 2 false negatives. It has a precision of 0.9976, recall of 0.9992, and an F1-score of 0.9984, with an alert volume of 2,513 against 2,509 true anomalies . The very low false positive count (6 across more than 83,000 normal blocks) means analysts would face no wasted alerts, which is the ideal profile for a SOC where alert fatigue is the main concern.

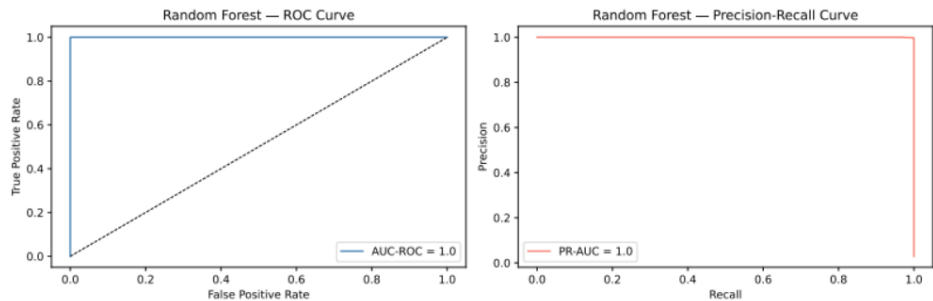


Figure 5.10: Random Forest ROC and Precision-Recall Curves (Oversampling)

From fig 5.10, both curves are perfect, with an AUC-ROC of 1.0 and a PR-AUC of 1.0, the ROC curve rising almost vertically to the top-left corner and the precision-recall curve holding precision at 1.0 across nearly the entire recall range. Because

PR-AUC remains the more reliable metric under the 2.93% class imbalance, its perfect value confirms that the model separates the anomalous and normal classes cleanly rather than benefiting from the easy majority class. This indicates that oversampling had full anomaly coverage without inflating false alarms.

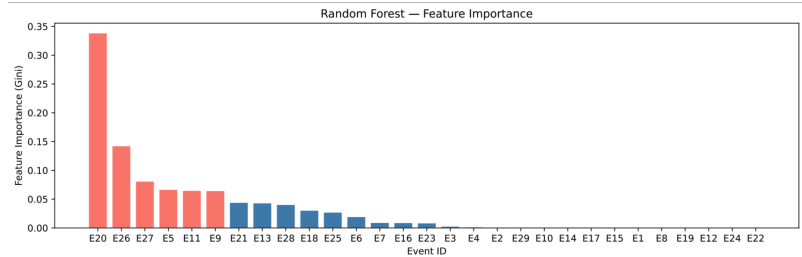


Figure 5.11: Random Forest Feature Importance

This figure shows that E20 accounts for around 34% of total impurity, with E26 following with 14%, and E27, E5, E11, and E9 forming the next group. The results suggest that the majority of the signal that will help to classify anomalies in HDFS_v1 will come from a small number of event-count features.

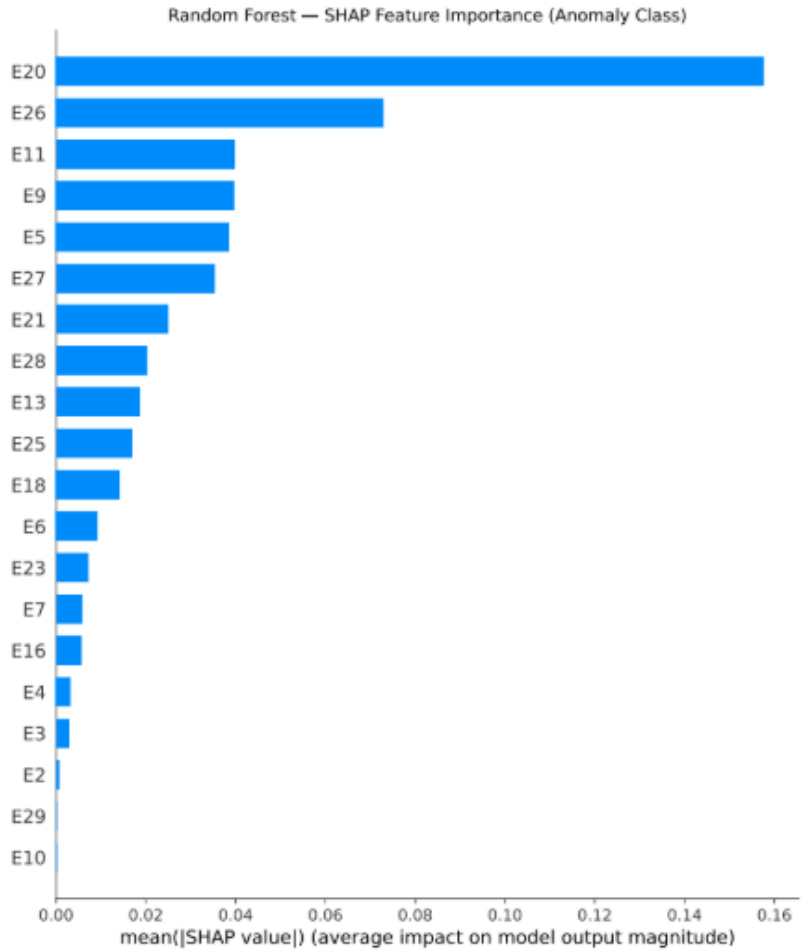


Figure 5.12: SHAP Feature Importance of Random Forest Model

Figure 5.12 refers that the SHAP analysis reinforces the Gini feature importance and the ranking lists E20 and E26 as the leading features $\sim 34\%$ of the total impurity reduction for E20 with event E26 contributing 14%. Next event E27, E5, E11, and E9 contributing between 6% and 8% each. This concentration of predictive signals in a few event-count features is in line with the Jensen-Shannon distance analysis of Section 4.2.1, which found that the event-count features E26 and E5 have the highest distributional difference between normal and anomalous blocks (JSD = 0.546 and 0.502, respectively) and that E20, though not as extreme in terms of JSD, is the single most decision-relevant feature of the classification boundary.

5.2.3 LSTM

The LSTM model was evaluated using sequence-based next-event prediction on the HDF5_v1 dataset. The model was trained only on normal logs and it learned the sequential behaviour of normal system events. During testing, anomaly detection was performed by checking whether the actual next event appeared within the top- g predicted candidates, where $g = 9$ was used throughout the experiment following the original DeepLog approach [1].

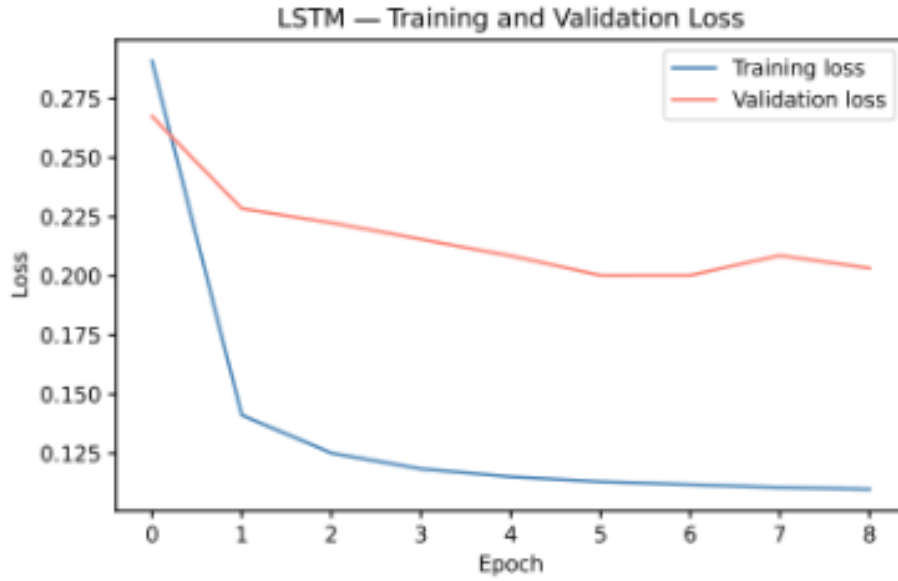


Figure 5.13: Training and Validation Loss Curves for LSTM Model

Fig 5.9 shows the training and validation loss curves show a rapid decrease during the first epoch, then it becomes almost flat in the next epochs, which indicate the fast initial learning by the model. The training loss goes to 0.12 and the validation loss tends to 0.20. The differences between the curves show some overfitting, but not too much. Overall, the convergence of the curves is relatively stable, suggesting

that the model is good in capturing the underlying patterns in the data with only minor differences between the training and validation results.

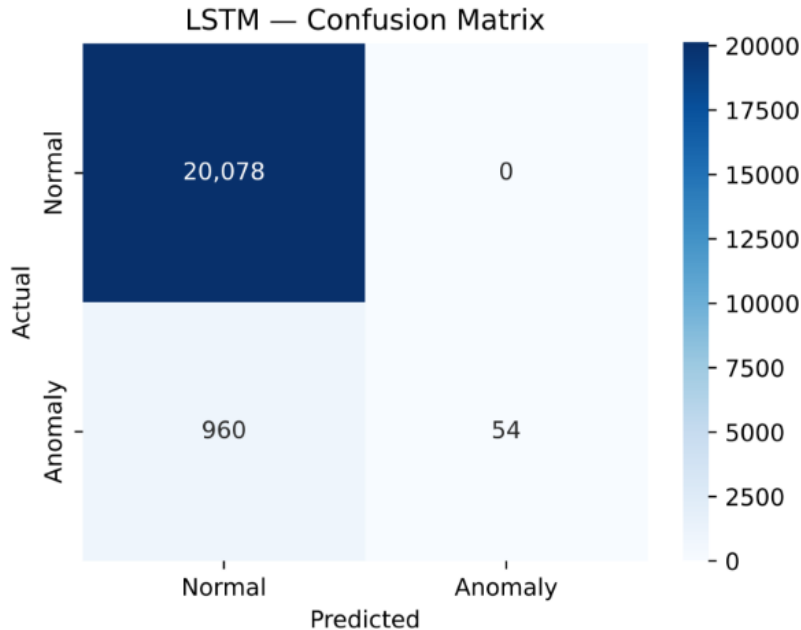


Figure 5.14: Confusion Matrix for the DeepLog-Style LSTM Model

Figure 5.14 shows is the confusion matrix that shows the model produced 20,078 true negatives, 0 false positives, 960 false negatives, and 54 true positives. From these results, the model achieved an accuracy of 95.45%, precision of 1.0000, recall of 5.33%, and F1-score of 0.1011. Overall, the model generated only 54 alerts in total. The model missed many anomalous blocks, which explains the very low recall. Therefore, the model has generted very few alerts, but it failed to detect many anomalies.

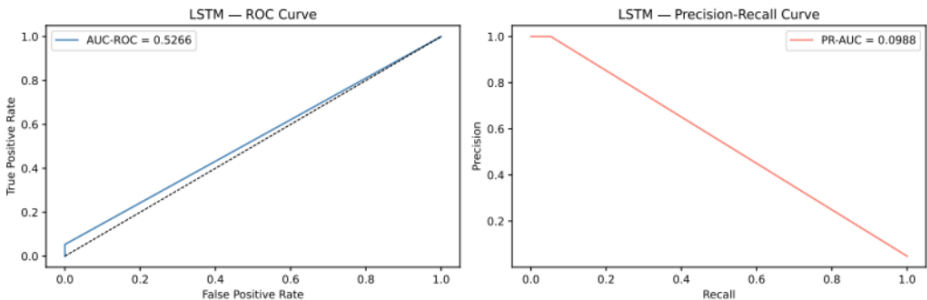


Figure 5.15: ROC and Precision-Recall Curves for LSTM model

Fig 5.15 depicts the ROC-AUC score was 0.5266, and the PR-AUC score was 0.0988. These values show that the model had limited ability to separate normal and anomalous blocks. Although the model did not produce false alarms, it was not strong

enough at detecting most anomalies.

A key limitation was identified during evaluation; out of the total 86,260 test blocks, only 21,092 blocks were evaluable by the LSTM model. It is because the model required a minimum sequence length of 20 to create sliding windows. Blocks shorter than 20 events could not generate predictions and were therefore excluded from evaluation. As a result, many anomaly samples were excluded before prediction even occurred. This explains why the model achieved very high precision but poor recall. From a SIEM perspective, this is an issue as short abnormal log traces may have failures or threats that need to be resolved immediately. Despite this limitation, the model still demonstrated that sequential learning can identify strongly abnormal event transitions in structured system logs [1], [19], [27].

5.2.4 LSTM Autoencoder

Normal-only training + normal validation threshold

The LSTM Autoencoder was tested using two different threshold settings, obtained from analysing the validation set, and representing two different priorities of operation in a SIEM deployment context.

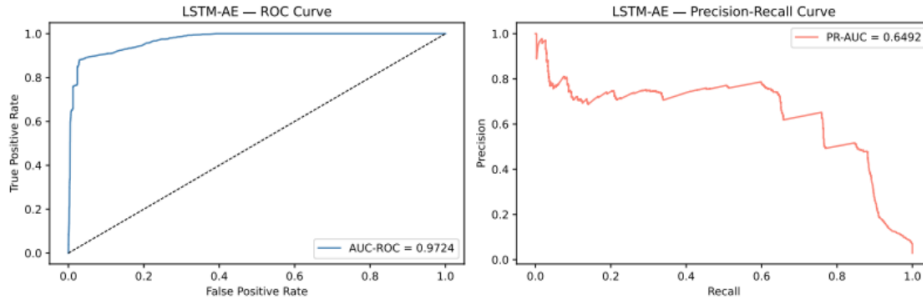


Figure 5.16: ROC and Precision-Recall Curve for the LSTM Autoencoder Model

Fig. 5.16 show the model's ROC curve results in an AUC-ROC of 0.9724, which is a good ranking capability, and the PR-AUC of 0.6492 is a moderate precision-recall tradeoff performance under the class imbalance of HDFS_v1.

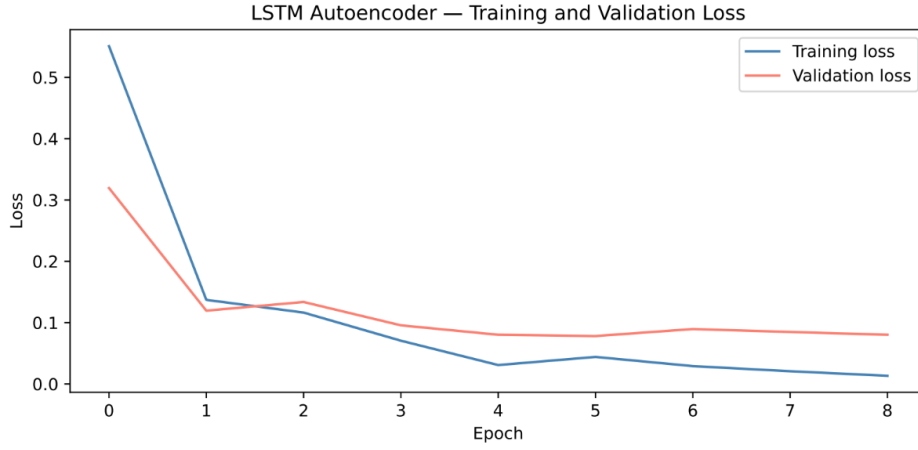


Figure 5.17: Training and Validation Loss for the LSTM Autoencoder Model

Fig 5.17 shows the training and validation loss curves of LSTM-AE that has clear divergence beginning around epoch 4, with training loss continuing to decrease toward near zero while validation loss plateaus around 0.08–0.09, which is an indicator of mild overfitting which shows the model’s reconstruction of normal sequences generalizes less to unseen data than the AE.

F1-Optimized Threshold (6.51)

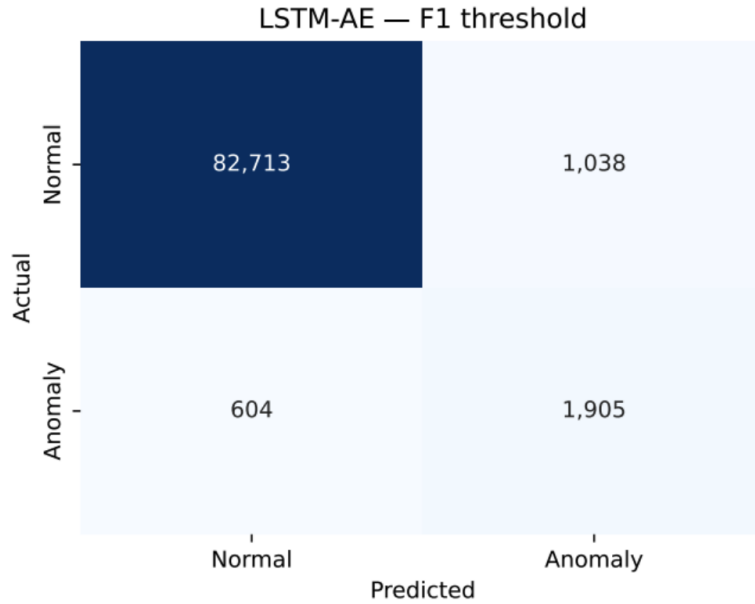


Figure 5.18: Confusion Matrix for LSTM-AE, F1-Optimized Threshold

Fig 5.18 shows that the confusion matrix, with the F1-optimized threshold of 6.51, the LSTM-AE obtained a precision of 0.6473, a recall of 0.7593 and an F1 score of 0.6988, which was the second highest F1 score among all the models evaluated. The false positive rate in this case was 0.0124, or 1,038 false alarms for 83,751 normal

blocks. Total alert volume was 2,943, with 1,905 true positives and 1,038 false positives and 604 genuine anomalies missed. This threshold does allow for a reasonably high recall with a reasonable burden of false positives for moderate sized SIEM deployments.

SIEM-Constrained Threshold (2.17)

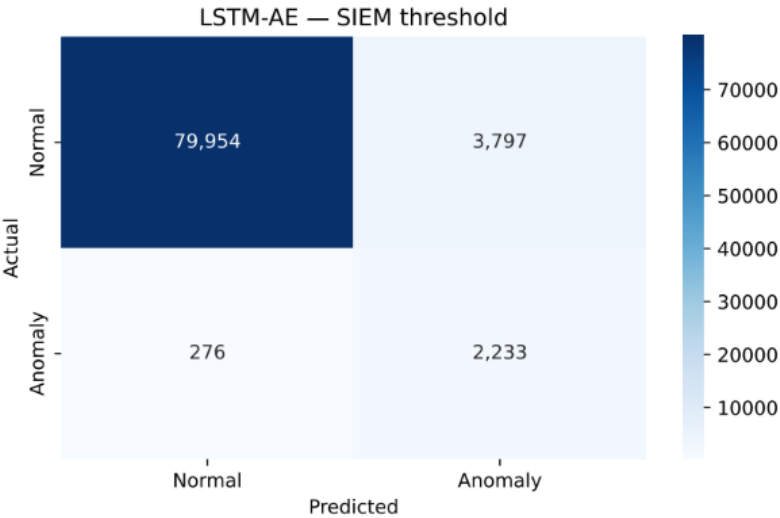


Figure 5.19: Confusion Matrix for LSTM-AE Model, SIEM-Oriented Threshold

Fig 5.19 shows the confusion matrix shows 79,954 TN, 3,797 FP, 276 FN, 2,233 TP. When the threshold was lowered to 2.17, as determined by the SIEM, to maximize the model's recall while keeping the FPR below an acceptable limit on the validation set, the model's recall rose significantly to 0.8900, or 2,233 true anomalies out of 2,509. But this improvement in sensitivity was at a high operational price: precision dropped to 0.3703 and FPR rose to 0.0453. Alert volume jumped to 6,030 which is twice the number of actual true anomalies, with a total of 3,797 false positives representing over 63% of the total number of alerts raised. This configuration gave the F1-score of 0.5230.

Normal-only training + balanced validation threshold

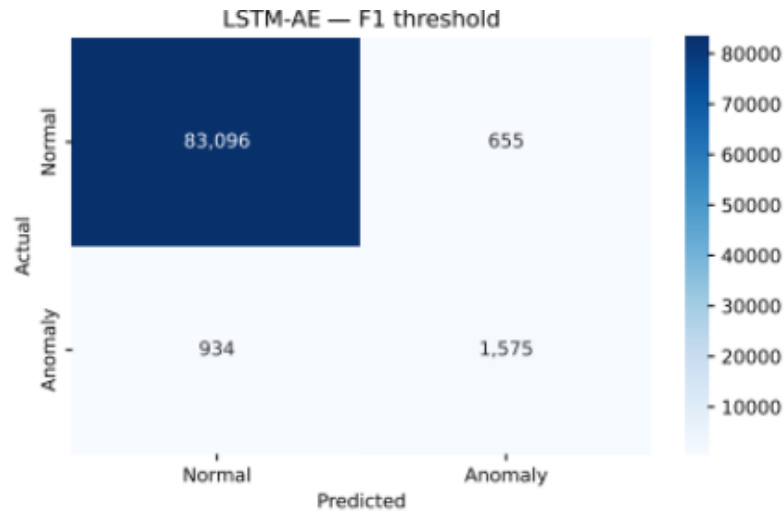


Figure 5.20: Confusion Matrix for LSTM-AE Model, F1-Threshold

From fig 5.20, the LSTM-AE correctly identified 83,096 of 83,751 normal blocks and detected 1,575 of 2,509 anomalies, resulting in 655 false positives and 934 false negative with a precision of roughly 0.71, a recall of roughly 0.63, an F1 score of roughly 0.66, and an alert volume of 2,230. The LSTM-AE model detects 655 false alarms and 1,575 anomalies, whereas the AE model detects 1,144 anomalies and 1,062 false alarms, demonstrating the trade-off between recall and precision between the two models, respectively. The Autoencoder provides more anomaly detection coverage from a SIEM perspective, but creates more false alarms which is a burden for the SOC analyst.

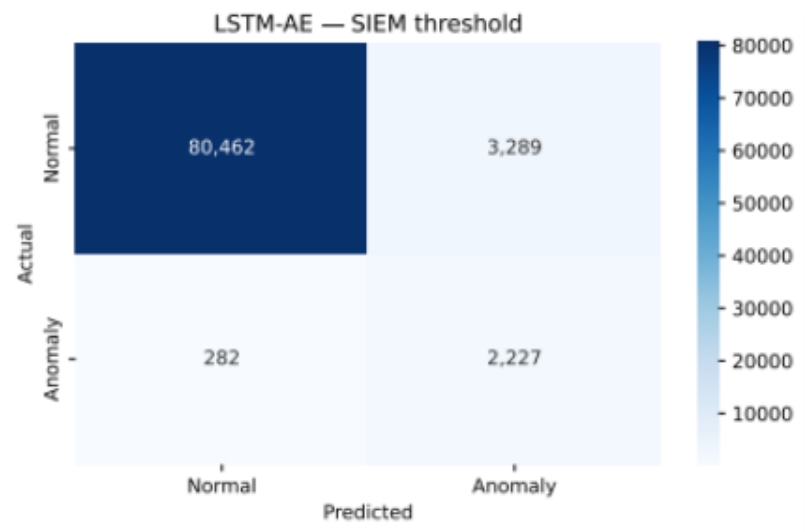


Figure 5.21: Confusion Matrix for LSTM-AE Model, SIEM-Oriented Threshold

Fig 5.21 shows the confusion matrix of LSTM-AE , SIEM- Threshold on balanced

validation. Of 2,509 anomalies, 1,575 were detected and 934 missed, with 655 false positives across 83,751 normal blocks.

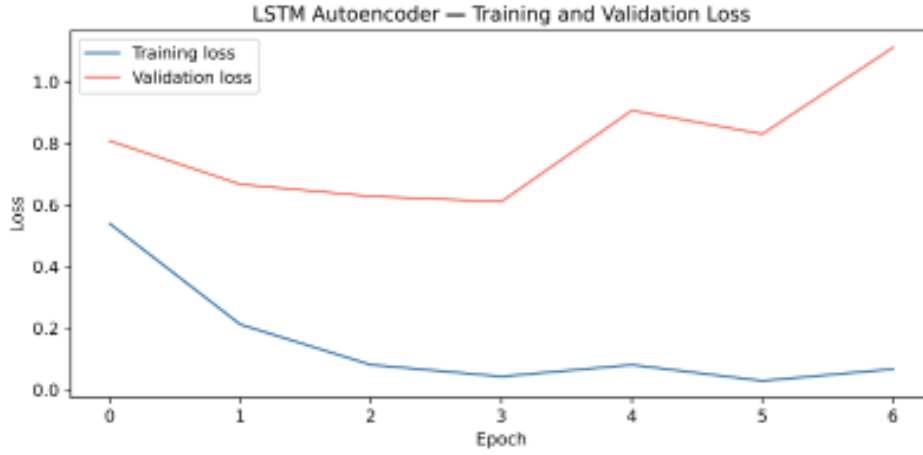


Figure 5.22: Training and Validation loss curves for LSTM-AE Model

From Figure 5.22, shows clear overfitting with such a huge gap. It is because the reconstruction error on unseen data could not flag anomalies properly.

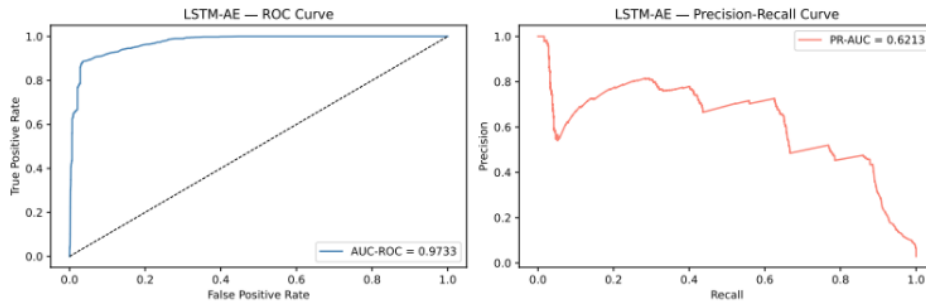


Figure 5.23: LSTM-AE ROC and PR curve

Figure above shows high AUC-ROC of 0.9733 indicating model's ability to differentiate between normal and anomalous blocks quite strongly. However, at low recall, precision is very unstable, indicating the model's struggles to precisely catch more anomalies.

5.3 Final Design Adjustments

This section describes the threshold selection decisions made for the models based on validation set. Table 5.5 provides the threshold values under Condition 1: normal validation and Conditon 2: balanced validation .

Model	Threshold Type	Condition-1 Value	Condition-2 Value
AE	F1-optimized	0.0059	0.000000
AE	SIEM-constrained	0.0059	0.006348
LSTM-AE	F1-optimized	6.5109	11,1727
LSTM-AE	SIEM-constrained	2.1703	3.4915

Table 5.5: Threshold Values of AE and LSTM-AE with Conditions from Table 4.6

5.3.1 Autoencoder without validation set balanced, F1-optimized threshold

For the Autoencoder, both conditions produced very close results: Precision of 1, Recall=0.4552, F1=0.6256, FPR of 0, with F1-optimized and SIEM-constrained threshold being the same, 0.005929. This showed how cleanly the model’s reconstruction errors for normal and anomalous blocks are so clearly separated that the best threshold is constant regardless of the validation set being balanced or not. Tackling the class imbalance did not really affect this model’s performance, only because the Condition 1 showed faster inference throughput (16,780 vs 11,890 blocks per second). However, most anomalies go undetected since it maintained a very low Recall of 45.52%.

5.3.2 LSTM-AE without validation set balanced, F1-optimized threshold

For the LSTM Autoencoder, it achieved the most balanced result on the F1-optimized, natural mixed validation set, with F1 = 0.6988, recall = 0.7593, precision = 0.6473, FPR = 0.0124, and 2,943 total alerts. The SIEM-orientated threshold showed the trade-off of getting higher Recall, with it rising to 0.89 but Precision dropped to 0.37 and alert volume soared to 6,030, with more than 63% of alerts being false positives. Such level of noise is what causes alert fatigue to SOC analysts. Hence, F1-optimized threshold is the better setting, giving around 1 false alarm every 7 alerts but still detecting most anomalies correctly.

5.3.3 Random Forest with Oversampling

For the Random Forest, the one with training data oversampled produced most optimum results, with F1 = 0.9984, recall = 0.9992, precision = 0.9976, FPR = 0.0001, and only 6 false positives out of 83,751 normal test blocks. In contrast to the balanced undersampling approach, which gave 55 false positives, this model learned normal behaviour completely by learning a better decision boundary. Alert

fatigue for analysts is minimized since 2513 alerts for 2509 true anomalies is quite good performance. Analysts are not burdened with unnecessary alerts and it shows how class imbalance when handled during training affects operational performance, for this dataset.

5.3.4 LSTM

For the LSTM, anomaly detection follows the DeepLog top-g rule of Du et al. [1], with a candidate set size of $g = 9$ applied throughout testing without needing validation testing, hence no further threshold tuning. A candidate grid of 9 evenly-spaced percentile-based values spanning the validation set score distribution was examined during the phase of model development. Since it has been demonstrated before in the report that anomalies in this dataset manifest mostly as frequency deviations rather than unusual sequences, and that it takes at least 20 events for the LSTM to make predictions, therefore this model is not suitable for this dataset.

Model	Condition	Precision	Recall	F1	FPR	Alert Vol.	Throughput blk/s
Random Forest	Oversampling	0.9976	0.9992	0.9984	0.0001	2,513	160,724
Autoencoder	Natural val.	1.0000	0.4552	0.6256	0.0000	1,144	16,780
LSTM-AE(F1-thres.)	Natural val.	0.6473	0.7593	0.6988	0.0124	2,943	24,712
LSTM	Fixed $g = 9$	1.0000	0.0533	0.1011	0.0000	54	10,052

Table 5.6: Overall Best Results obtained from class imbalance handling conditions

Table 5.6 shows Random Forest obtains best result on oversampling, LSTM-AE and AE on natural validation.

5.4 Statistical Analysis

As stated in Chapter 4.2.1, the severe class imbalance of HDFS_v1(2,509 true anomalies among 86,260 test blocks) suggest that accuracy and AUC-ROC are unreliable as primary comparison metrics, since both are inflated by the large, easily-classified normal majority [13], [21]. PR-AUC is therefore the more informative threshold-independent summary here, as it characterizes performance directly on the minority anomalous class [21]. On this metric the RF leads at 1.0000, followed by the AE at 0.9859 and the LSTM-AE at 0.6492. The LSTM model was excluded from pairwise statistical comparison because it was evaluated on a different subset of 21,092 blocks rather than the full 86,260-block test set, making direct comparison of error counts invalid. Results for the three comparable pairs are presented in Table 5.3.

To test whether these performance gaps reflect genuine paradigm-level differences rather than sampling variation, McNemar’s test was applied to each comparable model pair, a paired test suited to binary classification under class imbalance that makes no normality assumption [13], [21]. The standalone LSTM was excluded, as it was evaluated on a different subset (21,092 blocks) rather than the full test set, making its error counts non-comparable. Results for the three comparable pairs appear in Table 5.3.

Model Pair	McNemar Statistic	p-value	Interpretation
AE vs RF(Oversampling)	1,369.00	< 0.0001	Statistically significant
AE vs LSTM-AE	1799.00	< 0.0001	Statistically significant
RF(Oversampling) vs LSTM-AE	112.60	< 0.0001	Statistically significant

Table 5.7: Pairwise McNemar’s Test Results Comparing AE, RF, and LSTM-AE Models

All three pairs are statistically distinct at $p \leq 0.0001$, well beyond the conventional $p \leq 0.05$ threshold. The large statistics for the AE pairings are driven by the AE’s 1,367 missed anomalies faults that both the RF and LSTM-AE detect producing asymmetry. The smaller statistic for RF vs. LSTM-AE reflects their more modest divergence. In every case the differences are attributed to genuine architectural and paradigm-level distinctions rather than statistical noise.

5.5 Comparisons and Relationships

Before comparing the models, an important distinction has to be noted. The ML Random Forest is a supervised classifier, the only model which learns both normal and anomalous data, whereas the other DL models are all semi-supervised, learning from only normal behaving blocks. Therefore, explicitly stated, Random Forest, is a classical supervised baseline rather than an outright competitor for the Deep Learning models. The asymmetrical design is intentional to address the shortage of labelled anomalies in operational environments, and how the more complex and expensive deep learning models may actually contribute without access to anomalous data. So, the comparison reflects genuine trade-offs for deployment.

5.5.1 Models Detection Quality

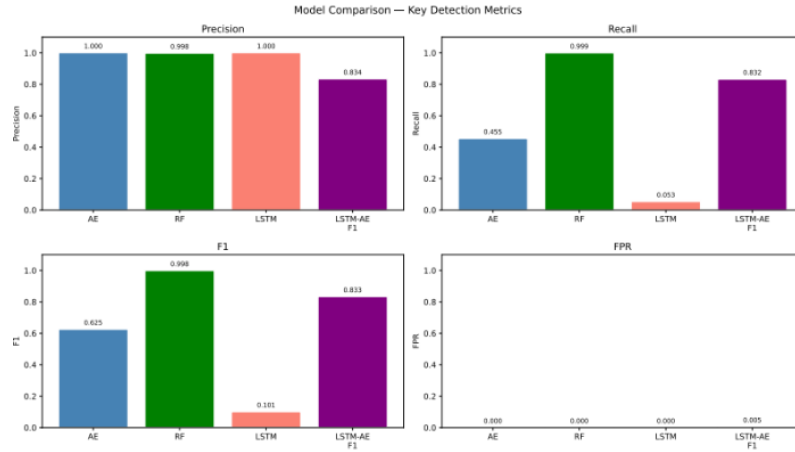


Figure 5.24: Comparison of Key Detection Metrics (Precision, Recall, F1, and FPR) across Evaluated Models

From the fig 5.24, the gap between the results of frequency-based and sequence-based approaches is noticeable. Random Forest with random oversampling performed best with an F1-score of 0.998, with the LSTM-AE close at 0.833, using the F1-optimized threshold. The Autoencoder gave an F1-score of 0.625, while the LSTM struggled, achieving only 0.101. One notable shift from earlier results is that the LSTM-AE now outperforms the AE, suggesting that pairing sequential reconstruction with careful threshold calibration gives better anomaly detection coverage than relying on frequency reconstruction alone.

These results make sense when looked at alongside the EDA findings from Chapter 4. From the 29 log event types, 15 shows a Jensen-Shannon distance over 0.1 between normal and anomalous blocks. That means the two classes differ most noticeably in how often certain events occur, not in what order they appear. Anomalies in HDFS_v1 tend to show up as unusual event counts, such as repeated error states or missing expected terminations, rather than as change in event sequences. This is exactly why models that pay attention to event frequencies performed well here, while the LSTM, which focuses on sequence patterns, had little to work with.

5.5.2 Operational Feasibility

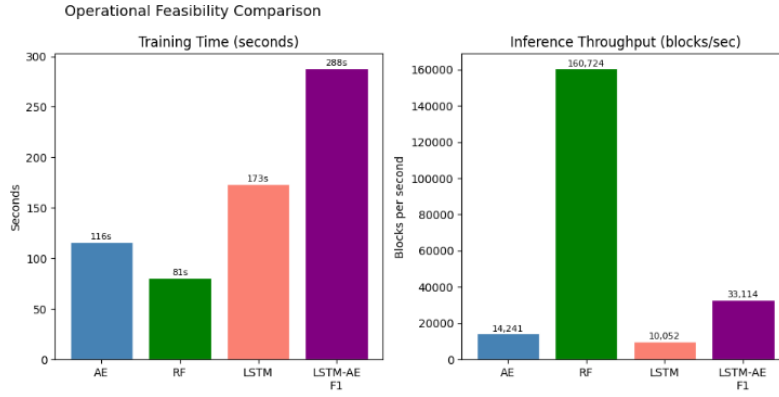


Figure 5.25: Operational Efficiency Comparison Detailing the Total Training Time (in seconds) and Inference Throughput (in blocks per second) for each model.

SIEM-style metrics, the operational metrics further validate the usage of simpler models for this specific dataset. The Random Forest model proved to be the most operationally efficient. It requires only 81 seconds for training and achieving an inference throughput of 160,724 blocks per second. The Autoencoder followed with a training time of 116 seconds and a throughput of 14,241 blocks per second. The recurrent models were the most computationally expensive, with the standalone LSTM requiring 173 seconds to train and achieving a throughput of only 10,052 blocks per second, while the LSTM-AE required 288 seconds to train and achieved 33,114 blocks per second. These metrics highlight that the RF model provides not only superior detection but also optimal computational efficiency for high-throughput logging environments.

5.5.3 Literature Comparison

Ref.	Model	Precision	Recall	F1-Score	Specificity
[1]	LSTM	≈ 0.946	≈ 0.959	0.96	—
[16]	AE	0.326	0.544	0.408	—
[16]	LSTM	0.909	0.526	0.667	—
[24]	LSTM-AE	0.714	0.764	0.737	—
[6]	RF	0.9964	0.9991	0.9978	0.9999
[6]	LSTM	0.9841	0.9869	0.9853	0.9995

Table 5.8: Reported Metrics from Prior HDFS LAD studies

Table 5.7 shows the results of the metrics used in previous HDFS log anomaly detection papers.

The F1 score of the Random Forest for oversampling is 0.9984 which is almost equal to the result obtained by Ali et al. [6] with an F1 score of 0.9978 using the frequency based block level features. This shows that classical supervised models are suitable for the frequency-driven anomaly structure of HDFS_v1.

LSTM achieved an F1 of 0.1011, DeepLog [1] recorded 0.96, and Ali et al. [6] recorded 0.9853. The gap is not due to any problem in implementation, but due to the structural factors mentioned in Section 5.2.3: HDFS v1 anomalies are mostly frequency based instead of sequence, and many short anomalous blocks were discarded due to its window length being below the minimum length. Kavlak and Sezen [16] also demonstrated that the performance of the LSTM sequence varies significantly with the windowing and evaluation method, with their result being 0.667.

The reconstruction-based models are consistent with most of the previous work. The Autoencoder’s 0.6256 is still higher than the 0.408 result of Kavlak and Sezen [16], which is due to the difference in the input (frequency-vectors instead of sequence-based reconstruction). The LSTM-AE’s performance of 0.6988 is similar to the 0.737 achieved by Bekkouche et al. [24].

Overall, the results suggest that HDFS_v1, is more strongly influenced by how the data is represented if its anomalies are sequence or frequency based than by model complexity .

5.6 Discussions

5.6.1 SIEM Deployment Recommendation

Based on the aggregate evidence across detection quality, statistical significance, and operational feasibility, the Random Forest is recommended for SIEM deployment on HDFS_v1-style structured log data. It achieves the highest performance across every measured metrics. With detection quality ($F1 = 0.9984$, $\text{recall} = 0.9992$), false alarm ($FPR = 0.0001$), training efficiency (81 seconds), how much logs is can process (160,724 blocks per second), and alert volume of (2,513 alerts for 2,509 true anomalies). When anomaly labels are unavailable for supervised training, a practical constraint in some operational environments, the LSTM-AE under the F1-optimized threshold is the recommended semi-supervised alternative, providing the

best detection coverage among the three deep learning models with a tolerable false positive burden. The LSTM model is not recommended for this task on this dataset; its near-random AUC-ROC and 5.33% recall make it operationally unusable for any meaningful security monitoring purpose. The Autoencoder, while offering uniquely perfect precision, leaves more than half anomalies undetected and would require significant threshold adjustment before deployment.

5.6.2 Alert Fatigue Implications

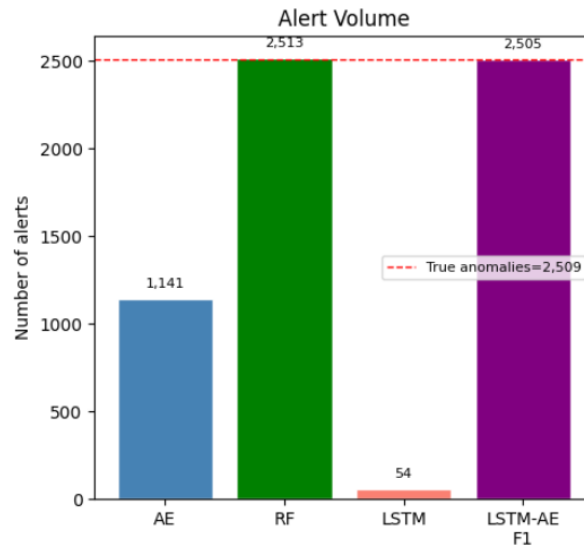


Figure 5.26: Total alert volume generated by each model relative to the ground truth

Alert fatigue is one of the most significant operational challenges faced by SOC analysts [14], [26]. The RF’s alert volume of 2,513 against 2,509 true anomalies represents an alert-to-detection ratio of approximately 1.002:1 for every 1,000 alerts an analyst investigates, 999 are genuine, and 1 is a false alarm. This is the operational ideal: maximal detection coverage with minimal analyst waste. The LSTM-AE at the F1-optimized threshold produces an alert volume of 2,505, a ratio of approximately 1.00:1, which means nearly every alert corresponds to a true anomaly though this comes at the cost of missing more anomalies than RF. The AE generates 1,141 alerts, all genuine, but misses more than half of true anomalies, representing a lack in detection, rather than alert fatigue. The LSTM generates only 54 alerts against 2,509 true anomalies, a ratio of approximately 0.02:1 missing over 95% of anomalies. These quantitative comparisons demonstrate that the SIEM-style evaluation framework adopted in this study incorporating alert volume and FPR alongside usual detection metrics captures operational consequences that standard benchmark reporting would entirely miss.

Chapter 6

Conclusion

This chapter concludes the thesis by summarising the key findings from the SIEM-style evaluation of the four log anomaly detection models, outlining the study’s contributions and limitations, and directions for future work.

6.1 Summary of Findings

This thesis evaluated four models for log anomaly detection, which are Autoencoder (AE), Long Short-Term Memory (LSTM), LSTM Autoencoder (LSTM-AE) and Random Forest (RF) on the HDFS_v1 benchmark dataset under a SIEM-style evaluation framework. Apart from standard evaluation metrics, we included False Positive Rate (FPR), alert volume, specificity, training time and inference throughput reflecting the constraints of a real Security Operations Centre (SOC) where excessive false alerts directly contribute to analyst alert fatigue [14].

One interesting finding of this study is that HDFS_v1 anomalies are primarily frequency-based rather than sequence-based. EDA confirms that multiple event types (E7, E8, E10, E12, E13, E14, E15, E29) have a mean occurrence of exactly zero in normal blocks, meaning any occurrence of these events constitutes a direct anomaly signal. The event E20 (“Unexpected error deleting block”) appears substantially more frequently in anomalous blocks than normal ones. This dataset characteristic directly explains the LSTM’s near-zero recall because it models event order transitions rather than frequencies and explains why frequency-based models (AE and RF) dramatically outperform it.

6.2 Contributions to the Field

This thesis is a comparative study. Its goal is not to propose a new detection model, but to evaluate existing ones in a way that reflects how they would perform in a real security setting. The main contributions are as follows.

1. **SIEM-oriented evaluation framework:** Most studies on the HDFS_v1 dataset report only detection-quality metrics such as precision, recall, and F1-score. This thesis relates these metrics in terms of SIEM environments, which are false positive rate, alert volume, specificity, training time, and inference throughput. These measures show the practical cost of running each model, not just how accurate it is [14], [17].
2. **Characterisation of the dataset before model selection :** Using Jensen-Shannon Distance over 29 event types, this thesis measures how differently each event occurs in normal and anomalous blocks. From the analysis, it was found that 15 of 29 event types have a JSD above 0.1, with a mean of 0.166, and that several event types appear only in anomalous blocks. This explains why frequency-based models succeed and the LSTM fails, and gives a data-driven reason for choosing one model over another instead of relying on assumptions from earlier work [22].
3. **Comparison of class imbalance techniques :** Rather than applying a single imbalance-handling method, this thesis evaluates each model under two experimental conditions to test how the choice of technique can affect performance. For Random Forest, balanced undersampling is compared against random oversampling. For the reconstruction-based models (Autoencoder and LSTM-AE), threshold calibration on a natural validation set is compared against calibration on a balanced validation set. The results show that the choice of imbalance technique has a different effects for example, oversampling reduced Random Forest’s false positives from 55 to 6 compared to under-sampling, providing empirical justification for technique selection rather than adopting a method by default, a comparison rarely made explicit in prior work [4], [6].
4. **Dual-threshold reporting for reconstruction-based models :** Each reconstruction model (Autoencoder and LSTM-AE) is reported at two operating points: one tuned for the best F1-score, and one set to meet a SIEM false-alarm budget. This shows that the same model can behave very differently

depending on the threshold — a trade-off that single-threshold reporting hides.

5. **Quantifying alert fatigue:** This thesis measures the ratio of alerts raised to true anomalies for each model. Random Forest raises 2,513 alerts against 2,509 real anomalies which is about 1.00:1. The LSTM-AE, under its SIEM threshold, raises 6,030 alerts which is around 2.40:1, flooding analysts with false alarms. Making this ratio explicit treats analyst workload as a real evaluation criteria.

6.3 Limitations

Some of our limitations are:

Single dataset scope: All experiments were done only on HDFS_v1. While HDFS_v1 is a widely used as benchmark [20][22], its anomalies are primarily frequency-based, which may not be representative of other log datasets where anomalies manifest as genuine sequence violations. The strong performance of RF and AE, and the failure of LSTM, are specific to this dataset structure and may not generalise to datasets such as BGL, where anomalies different [22].

Offline evaluation only: The evaluation was conducted in an offline batch setting using pre-labelled data. Real SIEM deployments have streaming log ingestion, concept drift as system behaviour evolves, and the absence of ground truth labels for threshold calibration. The absence of anomaly labels during training for the deep learning models reflects real operational conditions, but the threshold selection process still relied on labelled validation blocks, which would not always be available in practice [11][13].

Supervised requirement for Random Forest: Despite RF’s superior performance, its requirement for labelled anomaly examples during training represents a practical limitation in environments where ground truth labels are unavailable or expensive to obtain. The three deep learning models in this study can be trained on unlabelled normal data, giving them a deployment advantage in operational settings where anomaly labels are scarce [13][30]. The near-perfect results achieved by RF on HDFS_v1 should therefore not be interpreted as a general recommendation over unsupervised methods.

6.4 Future Work

In future, we plan on adding one additional dataset to test whether the current paradigm rankings hold beyond HDFS_v1. BGL is the natural candidate, as it represents a fundamentally different anomaly structure where anomalies primarily manifest as new event types rather than frequency shifts which would be expected to favour the LSTM over the AE [22][25]. Including complex datasets and working on new emerging models like transformers, LLM is also part of our future work.

Moreover, we plan to extend the evaluation to a streaming setting where models are evaluated on logs arriving in chronological order, thresholds are calibrated without access to future data, and concept drift is simulated by introducing new log templates mid-stream. This would more closely reflect the conditions under which a SIEM-integrated anomaly detector would operate and would allow assessment of whether the performance advantages observed in offline evaluation survive the additional complexity of real-time deployment.

6.5 Final Remarks

This thesis set out to answer whether deep learning-based log anomaly detection models are operationally useful in SIEM-style monitoring, not merely accurate on benchmark datasets. The findings reveal that: Random Forest is the strongest model across every metrics evaluated, including detection quality, training efficiency, inference speed, and alert generated. Among deep learning models, LSTM-AE provides the best balance of recall and false alarm control. The Autoencoder offers a reliable zero-false-alarm option at the cost of missing more than half of all anomalies. The LSTM, while theoretically well-motivated for sequential log data, is operationally insufficient on HDFS_v1 because the dataset’s anomalies are frequency-based rather than sequence-based. Hence, the LSTM results should be interpreted cautiously because evaluation was restricted to the subset of blocks that produced valid windows.

We need to understand the dataset first before choosing the models. A model that achieves perfect F1 on one dataset may generate different results on other. The SIEM-style evaluation style proposed in this thesis, incorporating FPR, alert volume, dual threshold reporting and operational throughput alongside standard metrics provides a more complete basis for that selection decision than accuracy-focused benchmarking alone.

Whether deep learning models will remain competitive with classical methods as

datasets grow more diverse and anomaly types more complex is an open empirical question. The findings of this thesis, consistent with Yu et al. [4] and Ali et al. [6], suggest that researchers should not assume that model complexity translates to operational superiority. The most useful anomaly detector is not the most architecturally sophisticated one, but the one whose detection mechanism best matches the anomaly structure of the logs it monitors.

Bibliography

- [1] M. Du, F. Li, G. Zheng, and V. Srikumar, “Deeplog: Anomaly detection and diagnosis from system logs through deep learning,” in *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2017, pp. 1285–1298. DOI: 10.1145/3133956.3134015
- [2] H. Guo, S. Yuan, and X. Wu, “Logbert: Log anomaly detection via bert,” in *Proceedings of the International Joint Conference on Neural Networks (IJCNN)*, Shenzhen, China, 2021, pp. 1–8. DOI: 10.1109/IJCNN52387.2021.9534113
- [3] S. Huang et al., “Hitanomaly: Hierarchical transformers for anomaly detection in system log,” *IEEE Transactions on Network and Service Management*, vol. 17, no. 4, pp. 2064–2076, Dec. 2020. DOI: 10.1109/TNSM.2020.3034647
- [4] B. Yu et al., “Deep learning or classical machine learning? an empirical study on log-based anomaly detection,” in *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*, Lisbon, Portugal, 2024, pp. 403–415. DOI: 10.1145/3597503.3623308
- [5] R. G. Albert, “System logs anomaly detection. are we on the right path?” *Applied Artificial Intelligence*, vol. 39, no. 1, 2025. DOI: 10.1080/08839514.2024.2440692
- [6] S. Ali, C. Boufaied, D. Bianculli, et al., “A comprehensive study of machine learning techniques for log-based anomaly detection,” *Empirical Software Engineering*, vol. 30, p. 129, 2025. DOI: 10.1007/s10664-025-10669-3
- [7] V.-H. Le and H. Zhang, “Log-based anomaly detection without log parsing,” in *Proceedings of the IEEE/ACM International Conference on Automated Software Engineering (ASE)*, Melbourne, Australia, 2021, pp. 492–504. DOI: 10.1109/ASE51524.2021.9678773
- [8] S. Lu, X. Wei, Y. Li, and L. Wang, “Detecting anomaly in big data system logs using convolutional neural network,” in *Proceedings of the IEEE DASC/PiCom/DataCom/CyberSciTech*, Athens, Greece, 2018, pp. 151–158. DOI: 10.1109/DASC/PiCom/DataCom/CyberSciTec.2018.00037

- [9] E. Alsalmi, A. Alhuzali, and A. Alhothali, “Log-based anomaly detection of system logs using graph neural network,” *Computers, Materials & Continua*, vol. 86, no. 2, pp. 1–20, 2025. DOI: 10.32604/cmc.2025.071012
- [10] A. Aziz and K. Munir, “Anomaly detection in logs using deep learning,” *IEEE Access*, vol. 12, pp. 176 124–176 135, 2024. DOI: 10.1109/ACCESS.2024.3506332
- [11] P. Himler, M. Landauer, F. Skopik, and M. Wurzenberger, “Anomaly detection in log-event sequences: A federated deep learning approach and open challenges,” *Machine Learning with Applications*, vol. 16, p. 100 554, 2024. DOI: 10.1016/j.mlwa.2024.100554
- [12] S. He, J. Zhu, P. He, and M. R. Lyu, “Experience report: System log analysis for anomaly detection,” in *Proceedings of the IEEE International Symposium on Software Reliability Engineering (ISSRE)*, Ottawa, ON, Canada, 2016, pp. 207–218. DOI: 10.1109/ISSRE.2016.21
- [13] M. Landauer, S. Onder, F. Skopik, and M. Wurzenberger, “Deep learning for anomaly detection in log data: A survey,” *Machine Learning with Applications*, vol. 12, p. 100 470, 2023. DOI: 10.1016/j.mlwa.2023.100470
- [14] S. Tariq, M. B. Chhetri, S. Nepal, and C. Paris, “Alert fatigue in security operations centres: Research challenges and opportunities,” *ACM Computing Surveys*, vol. 57, no. 9, pp. 1–38, Apr. 2025. DOI: 10.1145/3723158
- [15] W. Guan, J. Cao, S. Qian, J. Gao, and C. Ouyang, “Logllm: Log-based anomaly detection using large language models,” *arXiv preprint*, Apr. 2025, arXiv:2411.08561v5 [cs.SE]. DOI: 10.48550/arXiv.2411.08561
- [16] Y. Kavlak and A. Sezen, “A comparative study of deep learning models for log anomaly detection using the hdfs dataset,” in *Proceedings of the 5th International Conference on Informatics and Software Engineering (IISEC)*, Ankara, Türkiye, Feb. 2026, pp. 306–309. DOI: 10.1109/IISEC69317.2026.11418409
- [17] M. Khayat, E. Barka, M. A. Serhani, F. Sallabi, K. Shuaib, and H. M. Khater, “Empowering security operation center with artificial intelligence and machine learning—a systematic literature review,” *IEEE Access*, vol. 13, pp. 1–23, Jan. 2025. DOI: 10.1109/ACCESS.2025.3532951
- [18] X. Li, P. Chen, L. Jing, Z. He, and G. Yu, “Swisslog: Robust and unified deep learning based log anomaly detection for diverse faults,” in *Proceedings of the IEEE International Symposium on Software Reliability Engineering (ISSRE)*, Coimbra, Portugal, 2020, pp. 92–103. DOI: 10.1109/ISSRE5003.2020.00018

- [19] Z. Zhao, C. Xu, and B. Li, “A lstm-based anomaly detection model for log analysis,” *Journal of Signal Processing Systems*, vol. 93, pp. 745–751, 2021. DOI: 10.1007/s11265-021-01644-4
- [20] J. Zhu, S. He, P. He, J. Liu, and M. R. Lyu, “Loghub: A large collection of system log datasets for ai-driven log analytics,” in *Proceedings of the IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*, Florence, Italy, 2023, pp. 355–366. DOI: 10.1109/ISSRE59848.2023.00071
- [21] V.-H. Le and H. Zhang, “Log-based anomaly detection with deep learning: How far are we?” In *Proceedings of the International Conference on Software Engineering (ICSE ’22)*, New York, NY, USA: ACM, 2022, pp. 1356–1367. DOI: 10.1145/3510003.3510155
- [22] M. Landauer, F. Skopik, and M. Wurzenberger, “A critical review of common log data sets used for evaluation of sequence-based anomaly detection techniques,” *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 1354–1375, Jul. 2024, Article 61. DOI: 10.1145/3660768
- [23] W. Xu, L. Huang, A. Fox, D. Patterson, and M. Jordan, “Detecting large-scale system problems by mining console logs,” in *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*, 2009, pp. 117–132. DOI: 10.5555/3104322.3104329
- [24] M. Bekkouche, M. Meski, Y. Khodja, S. M. Benslimane, and E. Tronci, “Log-based anomaly detection using bilstm-autoencoder,” in *Proceedings of the International Conference on Networking and Advanced Systems (ICNAS)*, El Tarf, Algeria, 2025, pp. 1–6. DOI: 10.1109/ICNAS68168.2025.11298040
- [25] A. Oliner and J. Stearley, “What supercomputers say: A study of five system logs,” in *37th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN’07)*, Edinburgh, UK, 2007, pp. 575–584. DOI: 10.1109/DSN.2007.103
- [26] F. Jalalvand, M. B. Chhetri, S. Nepal, and C. Paris, “Alert prioritisation in security operations centres: A systematic survey on criteria and methods,” *ACM Computing Surveys*, vol. 57, no. 6, pp. 1–39, 2024, Article 142. DOI: 10.1145/3695462
- [27] W. Meng et al., “Loganomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs,” in *Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI-19)*, 2019, pp. 4739–4745. DOI: 10.24963/ijcai.2019/658

- [28] X. Zhang et al., “Robust log-based anomaly detection on unstable log data,” in *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '19)*, Tallinn, Estonia, Aug. 2019, pp. 807–817. DOI: 10.1145/3338906.3338931
- [29] S. Chen and H. Liao, “Bert-log: Anomaly detection for system logs based on pre-trained language model,” *Applied Artificial Intelligence*, vol. 36, no. 1, 2022. DOI: 10.1080/08839514.2022.2145642
- [30] A. Farzad and T. A. Gulliver, “Unsupervised log message anomaly detection,” *ICT Express*, vol. 6, no. 3, pp. 229–237, 2020. DOI: 10.1016/j.icte.2020.06.003